

Open Source, Hidden Costs: A Systematic Literature Review on OSS License Management

Boyuan Li, Chengwei Liu, Lingling Fan, Sen Chen, Zhenlin Zhang, and Zheli Liu

Abstract—Integrating third-party software components is a common practice in modern software development, offering significant advantages in terms of efficiency and innovation. However, this practice is fraught with risks related to software licensing. A lack of understanding may lead to disputes, which can pose serious legal and operational challenges. To these ends, both academia and industry have conducted various investigations and proposed solutions and tools to deal with these challenges. However, significant limitations still remain. Moreover, the rapid evolution of open-source software (OSS) licenses, as well as the rapidly incorporated generative software engineering techniques, such as large language models for code (CodeLLMs), are placing greater demands on the systematic management of software license risks. To unveil the severe challenges and explore possible future directions, we conduct the first systematic literature review (SLR) on 80 carefully selected OSS license-related papers, classifying existing research into three key categories, i.e., license identification, license risk assessment, and license risk mitigation. Based on these, we discuss challenges in existing solutions, conclude the opportunities to shed light on future research directions and offer practical recommendations for practitioners. We hope this thorough review will help bridge the gaps between academia and industry and accelerate the ecosystem-wide governance of legitimate software risks within the software engineering community.

Index Terms—OSS license management, Systematic literature review, OSS

I. INTRODUCTION

OVER the past few decades, the rapid development of open-source software (OSS) has been prompted by the spirit of technology sharing, offering significant flexibility and transparency. To avoid reinventing the wheels, an increasing number of developers choose to incorporate open-source artifacts into their projects across various levels of granularity to reduce development efforts. For instance, as reported by Census II, OSS constitutes 70-90% of any given piece of modern software solutions [1]. Approximately 70% of the code on GitHub actually consists of code clones from existing repositories [2], or even code snippets posted on Stack Overflow [3].

Every coin has two sides. The freedom of OSS does not negate the need for respecting the discipline in use. While

providing convenience to developers, OSS also introduces various legal risks in practical applications. To this end, software licenses have been widely adopted to regulate the disciplines and protect the copyright of software owners, and specifically, OSS licenses are proposed to balance copyright protection. Downstream users must adhere to the obligations of software licenses before enjoying the rights they are granted.

However, due to the complexity of software licenses and diverse ways of software reuse, it is still non-trivial to identify the potential legitimate risks when reusing OSS, and disputes over license violations during the development process are also common. Many big enterprises, such as Google, TikTok, and Microsoft have faced lawsuits and controversies related to software license violations. For instance, Google was found to have copied part of Java SE API code for its Android platform without a granted license [4], TikTok Live Studio was accused of violating GPL by incorporating code from OBS Studio without adhering to requirements [5], GitHub Copilot is also facing a class-action lawsuit for generating code without proper attribution to human-written code (e.g., name and copyright notice), potentially infringing on copyrights and violating various OSS licenses [6], [7].

To properly identify, manage, and mitigate the potential threats of these legitimate risks, both academia and industry have been devoted to extensively investigating the license risks in the open-source environment. Many related research works have been carried out, such as conducting empirical studies [8], [9], developing automated tools [10], [11], employing reviews and comparative analyses [12], [13], from various aspects, including but not limited to license incompatibilities [14]–[16], conflicts [17], inconsistencies [18], [19], violations [20]–[22], and compliance [23], [24], to facilitate better understandings of software licenses. However, despite extensive research efforts, the rapid development of software products and their corresponding licenses has introduced significant challenges in understanding licensing [25], [26], which is evidenced by the sparse application of research tools in industrial settings [27]. In parallel with academia's efforts, the industry has also been devoted to the standardization of open-source compliance practices to address the complexity of software licensing. For instance, the OpenChain Project [28] has provided structured guidelines and best practices to help organizations build robust open-source compliance programs [29]. The Software Package Data Exchange (SPDX) specification established a standardized framework for the structured presentation of license and copyright information [30], which serves as a foundation of software bills of materials (SBOMs) [31] and provides comprehensive transparency regarding software components

Boyuan Li and Chengwei Liu contributed equally to this work.

Boyuan Li is with DISec, NDST, College of Computer Science, Nankai University, China. (e-mail: boyuanli@mail.nankai.edu.cn).

Chengwei Liu is with Nanyang Technological University, Singapore (e-mail: chengwei.liu@ntu.edu.sg).

Lingling Fan (Corresponding author), Sen Chen, Zhenlin Zhang, and Zheli Liu are with DISec, NDST, College of Cryptology and Cyber Science, Nankai University, China (e-mail: linglingfan@nankai.edu.cn, senchen@nankai.edu.cn, liuzheli@nankai.edu.cn).

and their associated licensing details. Based on these, Software Composition Analysis (SCA) tools are further developed and incorporated to facilitate compliance by automating the identification of licenses and potential conflicts [32]–[35]. However, these efforts are still undermined by the poor understanding and implementation of software license restrictions [27].

To the best of our knowledge, no existing work has systematically reviewed and studied the workflow, as well as the capability of state-of-the-art (SOTA) approaches, of the current practice of OSS license management. In this paper, to bridge this gap, we collect research on license management and, guided by the functionalities of industry tools, propose a taxonomy that categorizes these works into three major objectives: license identification, license risk assessment, and license risk mitigation. Based on this, we conduct the first systematic literature review to delve into the management process of OSS licenses, based on which, we aim to unveil the persistent challenges and shed light on future research directions, for better ecosystem-wide governance of OSS licenses.

Our main contributions to this paper are as follows:

- **A comprehensive collection and review of OSS license management related research.** Based on the well-defined SLR method, we conduct a thorough analysis of 80 well-collected papers related to software license management.
- **A taxonomy of OSS license management research based on industrial practice.** We propose a taxonomy of OSS license management research guided by industrial tool functionalities and workflows, including license identification, license risk assessment, and license risk mitigation. This taxonomy reflects the alignment between academic research and industrial practice.
- **A discussion on challenges, opportunities, and recommendations for different stakeholders.** We extend a thorough discussion on existing challenges and limitations of existing license management, based on which we summarize the opportunities to propose possible future directions for license management. We also provide practical license management recommendations based on the current state of research for different stakeholders.

The remainder of this paper is organized as follows: Section II defines several important concepts and illustrates them with examples. Section III outlines the research methodology for paper collection and taxonomy construction. Section IV presents a comprehensive review of the selected studies, which serves as the basis for the in-depth discussion of key challenges, research opportunities, practical recommendations, and threats to validity in Section V.

II. BACKGROUND

As legal agreements that outline the conditions for the use, modification, and distribution of software, **OSS licenses** are commonly used to define the responsibilities and obligations of users, and provide legal protection for the copyright of developers to prevent their open-source software from being illegally shared and reused [36]. However, considering the continuous evolution of OSS licenses, their application, and

terminologies in the OSS ecosystem, we first clarify some key concepts in this paper to avoid misunderstanding.

• **License Proliferation:** It describes the phenomenon of the increasingly complicated legal environment for developers, companies, and users caused by the increasing complexity and prevalence of distinctive software licenses [37]. This occurs as licenses are customized to meet specific legal, ethical, or technological needs. Black Duck noted in their report that, among the codebases they audited in 2022 and 2023, 31% didn't contain standard licenses [38]. While these varied licenses provide tailored terms for software use and distribution, they also make compliance and interoperability more difficult, leading to confusion, higher legal risks, and potential misuse of software.

• **License Compatibility:** It refers to the ability of two software licenses to coexist without violating the terms of either [39]. This compatibility is essential for the legal reuse of software, especially when components or libraries under different licenses are combined in a single project. Understanding and ensuring license compatibility helps developers avoid legal pitfalls and ensures that software remains free and open for use and redistribution under specified terms.

• **License Non-Compliance:** License non-compliance in OSS involves failing to meet the specific conditions set by OSS licenses, such as proper attribution, releasing modified source code, or including the original license upon redistribution [40]. It often results from a lack of awareness or understanding of licensing terms, inadequate management of software assets, or the complexities involved in integrating multiple software components with different licensing requirements. Addressing license non-compliance is crucial for maintaining the legal and ethical integrity of software development and usage.

• **License Term:** License term (a.k.a. license clause [41], [42]) refers to rights granted and obligations imposed on users when using software [15], [43]. For example, the license term “redistribution” might appear as “distribute copies” or “redistribute” in licenses. Compared to license types, license terms provide a more granular framework for management.

• **License Exception:** License exceptions create exemptions from specific license conditions or extend permissions beyond the original terms [44]. A well-known example is the GPL linking exception [45], which allows programs to link to GPL-licensed libraries without fully complying with all GPL terms. Such exceptions function similarly to customized licenses but help mitigate license proliferation.

III. RESEARCH METHODOLOGY

To further understand the latest developments in OSS license management, we conducted an SLR on the subject and followed existing well-established guidelines [46]–[49]. Figure 1, adapted from the PRISMA flow diagram template [50], illustrates the complete workflow for paper retrieval and selection.

A. Search Strategy

To ensure the systematic and comprehensive collection of papers, we developed the following search strategies:

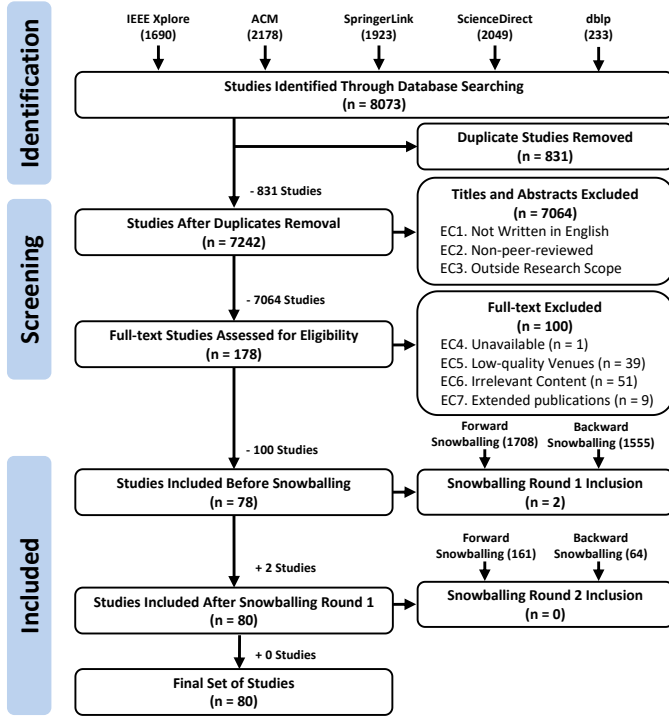


Fig. 1. Overview of literature retrieval and selection process

1) **Search Scope:** Our study aims to provide a comprehensive review of academic research on how developers manage project licenses. To this end, we seek to identify all relevant papers that focus on OSS license management as their primary research objective, such as selecting, tracking, altering, or complying with OSS licenses.

2) **Search Keywords:** After clarifying the search scope, we defined search keywords aiming at fully covering all relevant research papers within the search scope. The final keywords are classified into two groups as illustrated in Table I. *Group 1* limits the research coverage and consists of “oss” and its synonyms. Since the term “license” itself has various meanings and is widely used in industry and different fields, we incorporated a set of restrictive modifiers related to OSS to filter out irrelevant research initially. However, to avoid being overly restrictive, we selected inherently broad terms like “software,” “open source,” and “software supply chain,” which allowed us to cast a wide net and later refine the results through manual review. *Group 2* defines the focus of our research and consists of “license” and its variants, as we consider the presence of the term “license” to be a fundamental criterion for determining the relevance of a paper. To ensure the completeness of the collected papers, we included the wildcard character “*” in our search keywords. Finally, the search term is defined by the expression $g1 \text{ AND } g2$, where $g1 \in \text{Group 1}$ and $g2 \in \text{Group 2}$.

3) **Search Database:** Our search was conducted across five well-known digital databases: ACM Digital Library [51], IEEE Xplore Digital Library [52], SpringerLink [53], ScienceDirect [54] and DBLP [55]. It’s worth noting that different repositories and search engines utilize unique syntax rules and search fields for advanced querying. For instance, the

TABLE I
SEARCH KEYWORDS

Group	Keywords
1	floss; foss; oss; “open source”; software; “open source software”; “software supply chain”
2	licens*; license; licenses; licensed; licensing

ACM Digital Library allows the use of wildcards and supports keyword searches across abstracts, while DBLP does not support wildcards and limits searches to the paper title field. We thus tailored our search strategies to the specific rules of each database. The publication dates of the papers were limited to January 2000 through September 2024, aligning with the timing of the search. There were duplicates among the papers retrieved from different databases, so we merged the results to obtain a list of unique papers. After this, we collected 7,242 papers in total as the initial results.

B. Study Selection

1) **Paper Exclusion:** The papers obtained by keyword-based searching may introduce a significant volume of papers beyond our research scope. To refine the results, we implemented a set of exclusion criteria as below to filter out papers: **EC1. Papers not written in English.** Since English is the predominant language for international scholarly communication, English research can cover the widest range of innovative research and academic perspectives, thereby commanding greater acceptance and influence.

EC2. Non-peer-reviewed publications. We excluded papers that had not undergone rigorous peer review (e.g., theses, papers from arXiv, or unpublished manuscripts).

EC3. Out-of-scope papers. We excluded papers unrelated to our research scope (e.g., those concerning proprietary licenses or dataset construction), as well as license-related work in other fields or industries (e.g., driver’s licenses).

We first filter the collected papers by examining their titles and abstracts against EC1~EC3 and obtain 178 papers. For each paper, we downloaded the full texts and manually went through their content to further exclude irrelevant papers. Specifically, we adapted four additional exclusion criteria during the manual examination:

EC4. Studies whose full-text is unavailable. We ignored papers that we can only find an abstract rather than full text from all available sources (including the scholar databases and Google Scholar [56]), to avoid misunderstanding with only titles and abstracts.

EC5. Papers published in low-quality venues. Following the existing top-venue SLR [57], we excluded papers published in venues that were not included in the latest CORE ranking list to avoid insufficiently validated results [58], [59].

EC6. Papers with limited relevance to the research scope. Some papers were also found with limited relevance to our scope after going through the full text, so we also exclude them for better concentration. For instance, a paper may mention license compliance as motivation in its summary, but it does not actually address license management in the full text.

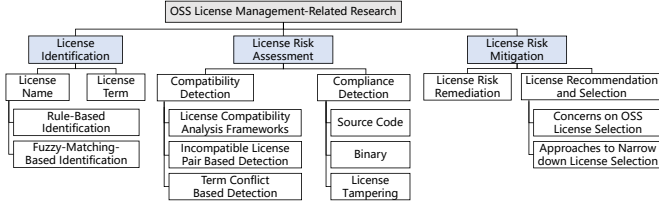


Fig. 2. The Overview of research works on OSS license management

EC7. Extended publications. We retained only the most extended version of a work. For instance, if a study was first presented as a conference paper and later extended into a journal article, we only keep the latter one with the most complete content.

In this step, a total of 100 papers were excluded: 1 for unavailability (EC4), 39 from low-quality venues (EC5), 51 due to irrelevant content (EC6), and 9 as extended publications (EC7). As a result, 78 papers were retained as our preliminary selection. The full-text screening was carried out by a three-member team comprising one doctoral candidate with two years of OSS-license research experience and two senior researchers, each with more than five years in the field. All three reviewers independently reviewed each paper to decide on its inclusion. As a result, inclusion disagreements occurred in only five of the 78 reviewed papers, and the average Cohen’s κ was 0.95, indicating a high level of agreement. The disagreements were finally resolved through discussion and consensus among the authors.

2) **Snowball Sampling:** To ensure no relevant research works were overlooked, we further conducted snowball sampling on the 78 collected papers. Specifically, for each round of snowball sampling, we recursively identified relevant papers that 1) cite it or 2) were cited by it, through Google Scholar [56], for each collected paper, until no more papers were included. Specifically, in the first round, we applied forward and backward snowballing to the 78 seed papers, retrieving 1,708 forward citations and 1,555 backward references. After merging the two sets and removing duplicates, 3,091 unique candidate papers remained. These candidates were then assessed against EC1~EC7. 70 papers met all criteria. Of these, 68 were already included in the seed set, so only two papers remained [60], [61]. A second round of snowballing based on these two papers returned no further eligible studies. Consequently, snowballing contributed two papers, bringing the total number of primary studies in our SLR to 80.

C. Paper Classification

We aim to develop a classification framework for the collected papers. However, the academic community has not yet established a widely accepted taxonomy in this area. Therefore, we draw on established industry practices, which offer mature workflows and widely adopted tools, to inform the structure of our taxonomy. To explore how industry solutions address OSS license issues, we focused on SCA tools, which are the most commonly used technologies for license management in practice. Guided by the Forrester

Wave evaluation report [104], a well-established and influential industry evaluation, we selected 12 leading SCA products with license-related capabilities. We then analyzed their publicly available resources, such as introductory materials, whitepapers, datasheets, and technical documentation, to identify the license-related functionalities implemented in these products.

The license-related capabilities of these tools are summarized in Table II. These functionalities encompass three key directions: license identification, license risk assessment, and license risk mitigation. Specifically, tools start by inspecting the source code and configuration files of user projects, and identify components and code snippets that are possibly third-party libraries. Next, they map the features (i.e., names, version numbers, code hashes, etc.) and query pre-constructed feature databases to confirm the suspicious third-party artifacts and relate them to known security issues or licenses. Then, they assess potential risks, and follow the SBOM standard to generate reports for users. Finally, some tools can provide suggestions or solutions for users to remediate identified issues.

We observed that the papers could also map onto these functional areas. However, the coarse groups failed to capture the nuanced differences among works. To this end, we first extracted the research objective of each paper, such as extracting license terms or assessing license compatibility. These objectives were then integrated into a twelve-category taxonomy shown in Figure 2. Three authors independently judged whether each paper belonged to specific categories, resulting in 960 classification records. Across all author pairs, there were a total of 66 disagreements, corresponding to classification differences in 21 unique papers. The average Cohen’s κ was 0.87, indicating substantial agreement. Disagreements were resolved through discussion and consensus. The complete classification is available at [105].

IV. LITERATURE REVIEW

In this section, we discuss the existing works on OSS license management by the most concerned functionalities, i.e., license identification, license risk assessment, and license risk mitigation. For each functionality, we discuss the existing works by their different objectives, granularity, and approaches in detail, as presented in Figure 2, and we also explore their relevance within the context of current industry perspectives.

A. License Identification

The gradual excessive reuse of software components in modern software development has fostered various needs for adopting software licenses. Consequently, various software licenses have been proposed to fit the newly emerged scenarios of software reuse, leading to the challenging identification of software licenses. Generally, the objectives of license identification experienced a migration from license names to detailed license terms, as well as changes in adopted techniques. Table III presents the timeline of existing research works.

1) **License Name Identification:** The primary purpose of license name identification is to determine which license governs a given software scope, such as repositories, folders, files, or code snippets, to facilitate follow-up risk assessment.

TABLE II
OVERVIEW OF FUNCTIONALITIES OF INDUSTRIAL LICENSE MANAGEMENT TOOLS

Industrial Tools	License Identification	License Risk Assessment		License Risk Mitigation			Publicly Accessible Information Sources
		Compatibility	Risk Profile	Attribution Notice	Remediation Options	License Selection	
Sonatype [32]	✓		✓	✓	✓		[62]–[64]
Black Duck [33]	✓	✓	✓	✓	✓		[65]–[69]
Mend.io [34]	✓	✓	✓	✓			[70]–[75]
Veracode [76]	✓		✓		✓		[77], [78]
Snyk [79]	✓		✓				[80], [81]
Checkmarx [82]	✓		✓		✓		[83]–[85]
Revenara [86]	✓		✓	✓	✓		[87]–[89]
JFrog [90]	✓					✓	[91], [92]
Palo Alto Networks [93]	✓		✓		✓		[94]
GitHub [95]	✓					✓	[96], [97]
GitLab [98]	✓	✓	✓				[99], [100]
Aqua Security [101]	✓	✓	✓				[102], [103]

Existing research on identifying OSS license names can be divided into two major strategies: 1) Rule-based identification, which relies on predefined rules to capture identical information from license text to distinguish different OSS licenses, and 2) Fuzzy-matching-based identification, which directly distinguishes OSS licenses by their textual similarity with known licenses. In this section, we introduce the development of these two main strategies in detail.

• **Rule-Based Identification:** ASLA [106] and FOSSology [11] are the first to introduce the capability of identifying software licenses by adopting regular expression templates to match the textual declaration of software licenses. As the pioneer prototype, although ASLA only supports the identification of 37 different types of software licenses, it proposed an extensive framework that allows for the expansion of regular expression templates. Similarly, Nomos, the license identification tool integrated into FOSSology, employs short seed expressions to extract areas of interest, standardizes these areas, and scans larger text sections to map to licenses. Until we drafted this paper, the rule-based templates in Nomos [107] supported the identification of over 1,700 licenses.

However, as pointed out by German et al. [10], these tools suffer from potential inaccuracy due to various scenarios of license declaration, such as mixing with source code, replacing with URL links, intended customization, and condition altering. To this end, Ninka [10] addressed these challenges with a sentence-based matching algorithm, which utilizes filtering keywords, equivalence phrases, sentence-token expressions, and license rules, to strengthen the accuracy and robustness of rule-based matching for 112 popular licenses. To further enhance the scalability of Ninka, especially when new licenses are proposed, Higashi et al. [108], [109] proposed automated solutions to cluster license statements from unknown licenses and extract sequential patterns to facilitate the automated generation of license matching rules.

Apart from license text, license names are also embedded with rich information for identification. Especially, relying on license names logged in the metadata of third-party libraries is most commonly used to retrieve license information by SCA tools. To this end, Wu et al. [9] proposed an extraction and algorithm based on AC (Aho-Corasick) automaton [110] and regular expression matching to precisely identify the standardized licenses referred by variations of license names.

TABLE III
A SUMMARY OF LICENSE IDENTIFICATION RESEARCH

Granularity	Method	Tool or First Author	Year
License Name	Rule-Based Identification	Nomos [11]	2008
		ASLA [106]	2009
		Ninka [10]	2010
		Higashi et al. [108]	2019
		Higashi et al. [109]	2023
		Wu et al. [9]	2024
	Fuzzy-Matching-Based Identification	Di Penta et al. [111]	2010
		LChecker [112]	2010
		Monk [11]	2015
		SORREL [113]	2021
		Wolter et al. [114]	2023
License Term	-	Vendome et al. [115]	2017
		FOSS-LTE [116]	2017
		DIKE [17]	2023
		LiDetector [15]	2023
		LiSum [117]	2023
		LiResolver [118]	2023

• **Fuzzy-Matching-Based Identification:** Although rule-based approaches have been demonstrated to be effective at license identification, their accuracy highly relies on the quality of pre-defined rules. It is challenging to define rules to perfectly fit all new-emerging licenses, especially under the context of license proliferation. For example, as revealed by Higashi et al. [119], the strict regular expression patterns often misclassified licenses when confronted with minor spelling errors or variant notations. In this context, fuzzy matching methods were introduced, utilizing text similarity techniques and machine learning classifiers to perform generalized matching across the license text.

For instance, FOSSology 3 integrated a new license identification tool, Monk [11], [120] to empower fuzzy license identification by matching licenses by text similarity. In this case, users can easily support new licenses by simply including their textual content for similarity analysis. Similarly, SORREL [113], a license management plug-in for IntelliJ IDEA, proposes a hybrid approach for identifying 16 popular licenses in Java projects, by introducing both machine learning classifier and Sørensen-Dice similarity coefficient, to further facilitate the detection of license incompatibilities.

Some researchers also rely on the matching of source code to trace the original OSS licenses. Di Penta et al. [111] conducted fuzzy-matching to the source code of jar archives.

They extracted textual information and bytecode of classes and queried the search engine to retrieve similar code, as well as their licenses. Zhang et al. [112] utilizes Google Code Search service to check whether a local file exists in an OSS project, and therefore, identify its original licenses.

Moreover, licenses may also be declared in different entities within a project, ranging from source-code comments, metadata files, to dedicated files such as *LICENSE*, *README*, or *COPYING*. Although such dispersion does not, in itself, hinder identification tools from extracting licenses, which typically rely on pattern-matching heuristics and are therefore indifferent to the specific textual format, it still brings difficulties. Wolter et al. [114] found that many projects exhibit inconsistencies in license declarations for the same component across these entities. It highlights the limitations of current tools, which struggle to reconcile licenses across diverse project entities accurately.

2) **License Term Identification:** The existing study [121] has shown that developers often have significant doubts and misunderstandings about existing licenses, highlighting a disconnect between their needs and the current licensing landscape. As a result, a multitude of new licenses have been created to address concerns raised by different stakeholders, leading to license proliferation. In particular, the prevalence of license customization [15], [122], challenges not only the identification of OSS licenses, but also the risk assessment after identifying the various OSS licenses. To this end, some researchers turned to inspect the detailed license statements to identify license terms and assess possible risks on rights and obligations directly, regardless of which license they are.

As the fundamental of term-based conflict analysis, most existing works construct their own models of license terms and map text to these terms to facilitate the identification of popular licenses. FOSS-LTE [116] initially manually summarized 37 types of license term templates and utilized Latent Dirichlet Allocation (LDA) for topic modeling to discern abstract topics within the license texts. These topics are then matched to the extracted license terms using vector similarity measures, ensuring that each term is associated with the most representative topics. However, as discussed in Cui et al. [17], the pre-defined templates should incorporate complex terms with specific attitudes, which require lots of manual effort and could result in low coverage of license types. To solve this, they proposed DIKE, which focused on the identification of 12 common license terms with inferences of the attitude for each license term, offering greater flexibility in identifying more licenses than FOSS-LTE. By modeling 3,256 licenses, DIKE constructed a *License-Terms-Responsibility* knowledge database, and based on it, DIKE built a term extraction model based on Label Specific Attention Network (LSAN) [123] to facilitate the identification of license terms.

To further enhance the accuracy of license term identification, based on the existing term definitions and classifications from SPDX [124], LiDetector [15] leverages a probabilistic model and Probabilistic Context-Free Grammar (PCFG) to analyze and categorize customized license content into 23 specific terms. As a subsequent work, LiResolver [118] incorporates a fine-grained extraction of license entities, relations, and

attitudes to achieve detailed fixing of license incompatibilities. Moreover, to automate the extraction of license terms for better readability, they further proposed a more generalized tool LiSum [117], which leverages the correlation between tasks of license text summarization and license term classification to incorporate more information and makes the summarization of licenses more accurate.

As another common practice of modifying existing licenses for compatibility issues, Vendome et al. [115] focused on the identification of license exception statements. By studying the prevalent license exceptions in practice, they trained machine-learning-based solutions to automatically identify license exceptions, which make up the empty usage and adoption of OSS license exceptions in this area.

Industry Perspective: Most existing SCA tools heavily rely on their pre-built metadata databases for license identification. Specifically, they usually identify potential license risks by identifying the third-party libraries that are integrated first, then query their corresponding licenses by referring to the metadata databases. In this case, it is easy to overlook license-related information that is not maintained in their databases, such as licenses mentioned in code comments [15]. Among the SCA tools we surveyed in Section III-B, only Black Duck [68] claims to supplement its analysis by scanning and identifying licenses embedded in software artifacts.

Remark 1: There is a huge gap between academic research and existing practices in industrial tools on license identification. Unlike mainstream SCA tools that rely on third-party library metadata for license identification, academic researchers have proposed many works and tools to systematically identify licenses of third-party artifacts, for not only known TPLs but also source code and binaries. Moreover, academic researchers have also noticed the proliferation of customized licenses and tried to propose solutions, and there is an urgent need for industrial tools to incorporate these solutions for better license identification in the future.

B. License Risk Assessment

Typically there are two major legal issues related to software licenses, 1) software license compatibility, which refers to the ability of different software licenses to coexist in a single project, especially in the scenarios of integrating software artifacts, source code, binary files with different licenses from third-party; 2) software license compliance, which involves adhering to the terms and conditions of software licenses, and this is essential to avoid legal consequences and maintain good standing in the software community.

1) **License Compatibility:** With the widespread practice of software reuse in contemporary software development, it is common for software components from third parties, such as libraries, binaries, and code snippets, to be integrated into the same project to reduce efforts. However, this integration leads to an excessive increase in the risks of license incompatibility issues. To this end, many researchers have investigated the detection of incompatibility issues by various approaches. Table IV shows the timeline of these research works.

• **License Compatibility Analysis Frameworks:** The issue of license compatibility has already been recognized and

TABLE IV
A SUMMARY OF LICENSE COMPATIBILITY DETECTION RESEARCH

Method	Tool or First Author	Year
License Compatibility Analysis Frameworks	Nordquist et al. [125]	2003
	LChecker [112]	2010
	Carneades [126]	2011
	Kenen [24]	2012
	CBDG [23]	2014
	Ilo et al. [127]	2015
	Riehle et al. [128]	2019
	Harutyunyan et al. [129]	2023
Incompatible License Pair Based Detection	Sun et al. [130]	2024
	German et al. [60]	2009
	German et al. [131]	2010
	Mathur et al. [132]	2012
	Mlouki et al. [133]	2016
	SPDX-VT [14]	2017
	Sorrel [113]	2021
	Qiu et al. [134]	2021
	Moraes et al. [135]	2021
	Higashi et al. [22]	2022
	Makari et al. [136]	2022
	Antelmi et al. [137]	2024
Term Conflict Based Detection	Alspaugh et al. [138]	2012
	LicenseRec [139]	2023
	SILENCE [16]	2023
	LiDetector [15]	2023
	LiResolver [118]	2023
	DIKE [17]	2023
	Liu et al. [43]	2024
	Wu et al. [9]	2024

analyzed by researchers since very early time. Although there was initially a lack of consensus and sufficient data regarding license incompatibility, researchers have proposed analytical frameworks to capture the scenarios of license adoption and possible conflicts. Nordquist et al. [125] are the very first to propose tools that intercept the build process and identify essential dependencies to examine to identify the applied software licenses, therefore, flagging missing licenses or potential conflicts. Van Der Burg et al. [23] then refined the dependencies by tracing system calls during builds, they constructed a Concrete Build Dependency Graph (CBDG) to identify sources that are truly included in deliverables and examine license compatibility issues within them. Moreover, considering that developers gradually learn and reuse existing open-source software in their own projects, Zhang et al. [112] proposed LChecker, which utilizes Google Code Search service to check whether a local file exists in an OSS project and whether the licenses are compatible. German et al. [24] systematically discussed the challenges in license compliance, and based on that, introduced Kenen, a semiautomatic process for license incompatibility analysis for Java projects.

Some researchers also contributed to the formalization and modeling of license compatibility issues. Gordon [126] introduced a prototype legal application for analyzing OSS license compatibility issues based on Carneades, which modeled and formalized legal rules and reason possible license incompatibility issues by OWL. Ilo et al. [127] also proposed SWREL, an ontology-based semantic modeling approach to combine and consequently query information about software interrelationships across different ecosystems, based on these, they

managed to detect potential license violations with user dependencies. With the evolution of software supply chains, researchers have introduced new governance frameworks [128], prompting organizations and projects to adopt standardized license management processes [129], [130]. These frameworks primarily focus on establishing comprehensive SBOMs and strengthening audits of third-party components, thereby mitigating the risk of internal license conflicts arising from the integration of upstream software components.

• **Incompatible License Pair Based Detection:** With the growing awareness and adoption of OSS licenses, both academic and industrial researchers have contributed a lot to building the consensus understanding of incompatibility among mainstream licenses. Based on well-established incompatible license pairs, German et al. [60] investigated the potential license violations during the frequent code exchange between Linux and two BSD systems with Unix kernels (i.e., FreeBSD and OpenBSD). Similarly, Mathur et al. [132] evaluated the code migration on Google Code and examined their potential license compatibility issues. Particularly, due to the viral nature of copyleft, GPL family licenses are mostly concerned by researchers and industrial SCA products [69], [71], [99], [102]. German et al. [131] analyzed inconsistencies between the declared licenses of binary files and source code in the Fedora 12 operating system, to identify GPL-related compatibility issues. Mlouki et al. [133] investigated the licenses that are mostly used in mobile apps, and how these licenses, as well as potential license violations, especially GPL-related incompatibility, evolve over time. Higashi et al. [22] analyzed Docker images from GitHub and found that GPL-incompatible software packages are fairly common, with a significant portion of images exhibiting these issues.

As the proliferation of OSS licenses, apart from GPL licenses, researchers also gradually concentrated on compatibility issues introduced by other new-emerging licenses. Kapitsaki et al. [14] proposed the compatibility map, which largely expanded the scope of compatibility detection. They developed a relation graph for over 20 prevalent licenses, detailing features like link types, license upgrade options, and one-way compatibility. Moraes et al. [135] also utilized this graph to conduct an empirical analysis in JavaScript projects, they found that 24% multi-licensed projects introduced at least one license incompatibility issue. Following this principle, SORREL [113] is proposed to retrieve licenses from metadata and assess incompatibilities for Java projects by referencing predefined compatibility lists. Qiu et al. [134] also conducted an empirical study on the incompatibility issues hidden in the dependency relations among NPM packages based on a license compatibility graph of 17 popular licenses. Makari et al. [136] integrated license incompatibility information from multiple sources and constructed a matrix containing 1,476 license compatibility pairs (i.e., among approximately 40 licenses). Based on this, they conducted empirical studies on compatibility in the NPM and RubyGems ecosystem. Compared to Qiu et al. [134], they identified much more incompatibility issues due to higher license coverage. Similarly, Antelmi et al. [137] introduced the Software Heritage Analytics (SWHA) framework to look into the license compatibility issues, they

also followed the existing OSADL license checklist [140] to facilitate comprehensive analysis of license incompatibility among GitHub repositories.

• **Term Conflict Based Detection:** Due to the proliferation of customized licenses, license matrix-based detection is becoming less adequate for practical requirements. For instance, SPDX License List [124] indexed the popular licenses that are widely adopted in practice, which is far more than those maintained by any license compatibility knowledge. In this case, term-conflict-based detection is becoming the mainstream technique in license incompatibility analysis.

Some researchers focused on establishing a more detailed and comprehensive knowledge base of popular license compatibility based on term conflict. Xu et al. [139] considered 19 license terms to build a compatibility matrix [141], which contains one-way compatibility information for 63 types of licenses certified by FSF or OSI in their license recommendation tool LicenseRec. In their subsequent tool, SILENCE [16], they implemented a breadth-first algorithm imitating the search behavior of PyPI dependency resolution to get dependencies in projects and combined the compatibility matrix to detect license compatibility in the PyPI ecosystem.

Some researchers also directly examined license conflict based on detailed license terms instead of whole licenses. Alspaugh et al. [142] first identified key properties of OSS licenses, presented a license analysis scheme, and proposed the semantic parameterization analysis of open-source licenses to fulfill intellectual property requirements management for heterogeneously-licensed systems [143]. Moreover, to better explain the license conflicts for software architects, they further proposed a tool to simultaneously provide explanations, as well as the trade-offs for alternative licenses, in heterogeneously-licensed systems [138].

Xu et al. [15] deepened the idea of license modeling and introduced an automated term-conflict-based detection tool, LiDetector. Based on this, they further defined term conflicts in detail by considering different licensing scopes, i.e., parent-child license incompatibility [118]. Wu et al. [9] utilized LiDetector and conducted an empirical analysis of license usage across five major package management platforms (Maven, NPM, PyPI, RubyGems, and Cargo). The fine-grained, term-based detection method revealed that approximately 8% of cases exhibited license incompatibilities, primarily due to the violation of sublicensing [144] between GPL and permissive licenses. Cui et al. [17] further extended the incompatibility detection to a wider range of 3,256 licenses, they developed License Analyzer to automatically retrieve license terms and reasoning potential license incompatibility by *License-Terms-Responsibility* relationships. Liu et al. [43] enhanced term-conflict-based detection from the perspective of term framework, they conducted a term-level analysis of term labeling on 453 SPDX licenses. They summarized the inconsistencies in license terms from Choosealicense [145], TLDRLegal [146] and OpenEuler [147], and marked these SPDX licenses with refined terms and extended conflict relationships.

Industry Perspective: Academic studies on license incompatibility analysis focus more on the complex and context-aware models that aim to provide detailed analysis of license

TABLE V
A SUMMARY OF LICENSE COMPLIANCE DETECTION RESEARCH

Categories	Tool or First Author	Year
Source Code	Monden et al. [149]	2010
	Boughanmi [150]	2010
	An et al. [151]	2017
	Lotter et al. [152]	2018
	Baltes et al. [153]	2019
	Ragkhit. et al. [154]	2019
	Golubev et al. [155]	2020
	Serafini et al. [156]	2022
	Ciniselli et al. [157]	2022
	CODEIPPROMPT [158]	2023
	Kapitsaki et al. [159]	2024
	ModelGo [160]	2024
Binary	Molina et al. [161]	2007
	BAT [162]	2011
	OSSPolice [21]	2017
	Feng et al. [163]	2019
	OSLDetector [164]	2020
License Tampering	Di Penta et al. [165]	2009
	Di Penta et al. [166]	2010
	Wu et al. [18]	2015
	Vendome et al. [167]	2015
	Mlouki et al. [133]	2016
	Wu et al. [19]	2017
	Vendome et al. [168]	2017
	Reid et al. [169]	2023

interactions to comply with legal and ethical requirements, while industrial SCA tools typically focus on detecting basic license conflicts or flagging high-risk licenses, based on existing known knowledge. Especially, leading SCA tools [69], [71], [99], [102] focus more on license conflicts that could influence commercial usages by industry, such as the well-known viral copyleft licenses [148]. To achieve better license incompatibility detection, Mend.io [71] has adopted license compatibility graphs and tables as references to identify a broader range of license incompatibilities, while these pre-defined compatibility resources are relatively limited.

Remark 2: There is a huge gap in license incompatibility analysis between academia and industry, academic researchers have conducted numerous works to identify any possible license incompatibility (even if license terms are not detected in a standardized license), while industrial SCA tools mainly focus on identifying certain types of license incompatibility that could really lead to commercial losses. This could be due to the unawareness of rights and obligations in open-source communities. We believe there is still a long way for different stakeholders to reach consensus on the severity of license incompatibility.

2) **License Compliance:** Besides license incompatibility among licenses, detecting the overall compliance of all licenses before officially releasing is also vital for software providers. Table V presents the existing research on software license non-compliance detection.

• **License Non-compliance Detection on Source Code:** The most important concept in the context of license non-compliance lies in the definition of *derivative works*. Specifically, to what extent source code reuse can be identified as

derivative works, there is still no consensus so far. To bridge this gap, most research works rely on code clone detection techniques to distinguish the potential reuse of protected intellectual property of source code.

Monden et al. [149] were the first to convene expert programmers to empirically analyze source code reuse under the GPL license, they introduced metrics such as the Maximum Length of a Clone (MLC) and the Local Similarity Lower Bound (LSim), to facilitate the identification of license violations. Based on this, Golubev et al. [155] investigated the potential code borrowing among Java projects on GitHub and unveiled that 29.6% of these Java projects might be involved in potential code borrowing and 9.4% of them could potentially violate original licenses. Serafini et al. [156] integrated Software Heritage Project and introduced SWH-scanner to efficiently identify prior publications existing in user projects, to alert where it comes from and whether additional obligations shall be fulfilled before product shipment.

Apart from GitHub, developers also refer to online technical forums, such as Stack Overflow (SO), to learn and reuse existing solutions. However, such reuse could also lead to licensing challenges. When reusing code from Stack Overflow, many developers ignore its *CC BY-SA* requirements [170], and some also fail to comply with the original project's licensing when posting its snippets on the platform. An et al. [151] first conducted a large-scale empirical study on this phenomenon, revealing extensive bi-directional code reuse between the Stack Overflow platform and Android projects. However, nearly all instances of code reuse failed to adhere to the requirements of proper license declarations. Lotter et al. [152] conducted a similar investigation between Java projects and SO, revealing the prevalence of code reuse in these copying flows. They also found that a significant portion of reused code lacked proper attribution, further increasing the risks associated with merging such code. Baltes et al. [153] further conducted a large-scale empirical study and surveys to investigate to what extent developers are aware and follow the attribution requirement by SO, they found that almost one-half admitted copying code from SO without attribution and about two-thirds were not aware of the license of SO code snippets and its implications.

Moreover, since Stack Overflow also allows users to license their submitted content, Ragkhitwetsagul et al. [154] extended to the compliance of these user-defined licenses and noticed that many source code snippets that are under copyleft licenses, are submitted as Stack Overflow answers while being relicensed with permissive licenses. In this case, such non-compliant relicense could also risk downstream users and lead to potential legitimate issues.

As open-source projects increasingly involve diverse forms of code reuse and integration, determining whether a code-base constitutes *derivative work* may not be solely addressed through clone detection methods. Boughanmi [150] proposed a compliance analysis framework that summarizes the interconnection mechanisms between components, such as linking, fork, subclass, IPC, or plugin, while constructing the dependency graph of the software system. These interconnection types may influence the compliance assessment for derivative projects. Duan et al. [160] focused on the scenario of deep

learning projects, in which it involves interactions among diverse types of licenses and licensed materials for compliance checks. To this end, they developed ModelGo, a practical tool to handle complex interactions and audit potential legal risks in machine learning projects to enhance compliance and fairness.

With the gradual engagement of AI-based techniques in modern software development, it is unavoidable that LLMs that are trained from open-source data, can easily generate code that is originally from open-source projects to users, which makes it even blurred to distinguish the *derivative works* in the context of license compliance. To this end, some researchers have tried to unveil the potential risks behind generative software engineering. Ciniselli et al. [157] first proposed the concern that it is unclear whether the code generated by DL models trained on open-source code should be considered as new or as derivative work, with possible implications on license infringements. They investigated the extent to which DL models tend to clone code from their training set when recommending code completions. They found that Type-1 clones of training instances account for approximately 10% of short predictions, dropping to 0.1% for longer ones. To specifically evaluate the potential IP violation risk, Yu et al. [158] proposed CodeIPPrompt, which automatically evaluates the extent to which code language models may reproduce licensed programs. They found that most IP violation issues in generated code are because of the implicit inclusion of restrictive code in permissive and public-domain code repositories due to inconsistent licensing practices. To avoid potential legitimate risks, Kapitsaki et al. [159] systematically discussed the responsible model, as well as possible legitimate issues that could be introduced by potential code reuse, along the workflow of LLM-based code generation tasks.

- **License Non-compliance Detection on Binary:** Binary is another typical type of deliverables of software artifacts. Compared to detecting source code reuse, it is usually not that straightforward to detect third-party reuse in binaries, and such reuse is usually not accompanied by explicit license declarations, making it easy to overlook and challenging to detect potential non-compliance.

Molina et al. [161] were the first to propose using binary code cloning techniques for detecting unauthorized use of open-source projects in pirated programs. They transformed binary programs into Static Single Assignment (SSA) and then extracted expression trees as fingerprints for similarity matching. Besides reverse engineering, some researchers also map binaries back to source code to associate the binaries to their licenses for compliance assessment. Hemel et al. [162] proposed BAT to inspect string literals that are usually invariant during compilation, as cursors, to identify potential code clones from third-party binaries or source code. To further enhance the accuracy and efficiency of similarity analysis, OSSPolice [21] incorporated the inherent characteristic features of targeted binaries, such as string literals, constants, normalized classes, and exported functions, in the similarity comparison. By comparing to a well-constructed dataset of 132K OSS sources, OSSPolice revealed that over 40K apps potentially violate GPL/AGPL licensing terms. Feng et al. [163] extended the

similarity comparison by comprehensively including not only code features that are invariant during compilation and could vary between projects, but also file attributes of executable files, such as file names, internal names, company names, and legal copyright, from *VERSIONINFO* resource of PE files. However, these more general features could easily introduce false positives in code clone detection. To deal with this, Zhang et al. [164] proposed OSLDetector, based on internal cloning forest, which improves the efficiency of feature duplication between libraries, resulting in the optimal selection of clone detection candidate as the final result.

- **License Tampering:** The license tampering in OSS presents a significant challenge in identifying potential license risks. The widespread practice of reusing existing open-source code through direct copy-pasting often leads to a single code snippet being traced back to multiple open-source repositories when determining third-party reuse. However, many instances of such copying neglect the critical necessity of verifying the compliance requirements of the original licenses (i.e., missing necessary copyright and license notice, or directly relicensing).

To deal with this challenge, researchers are gradually concerned with the potential threats of license provenance changes. Di Penta et al. [165], [166] proposed an automated approach for license tracking. They extracted change information from version control systems as input, identified license content within these changes, and conducted empirical studies to summarize the evolving status of contributors, types of licenses, and copyright years across multiple open-source repositories. By examining such metadata information, researchers also unveil the original licenses and changes of libraries and code snippets in different ecosystems. Researchers have also extended such license consistency analysis to various scenarios, including the Debian project [18], Java repositories on GitHub [19], and the Android ecosystem [133]. License tampering also happens in version releases, Vendome et al. [167], [168] analyzed license changes in the version evolution of GitHub projects and found that many changes lack corresponding commit records or discussions, resulting in poor traceability. Reid et al. [169] further confirmed the prevalence of license omission significantly introduced license violations at a large scale by tracking the license altering history logged in World of Code, suggesting the necessity of regulations of license traceability for open-source reuse.

Industry Perspective: To identify potential license compliance risks in projects, the industry similarly emphasizes code tracing analysis. However, this domain rarely focuses on detecting the fulfillment of obligations specified by license terms. Instead, they focus more on identifying whether specific high-risk license types of interest to developers appear in the project. Additionally, with the rise of generative code, some leading SCA tools have realized potential compliance risks associated with generative code [66], [84]. Nevertheless, current detection methods remain limited, relying on component analysis of generated code to monitor compliance risks.

Remark 3: There is a gap between academia and industry in the assessment of license compliance. Unlike mainstream SCA tools, which focus solely on identifying high-risk li-

TABLE VI
A SUMMARY OF LICENSE RISK MITIGATION RESEARCH

Categories	Objective	Tool or First Author	Year
Risk Remediation	-	German et al. [8]	2009
		Hammouda et al. [171]	2010
		Scacchi et al. [172]	2012
		Viseur [173]	2016
		Vendome et al. [174]	2016
		Mlouki et al. [133]	2016
		findOSSLicense [175]	2019
		ALP [42]	2019
		Sorrel [113]	2021
		LicenseRec [139]	2023
		DIKE [17]	2023
		LiResolver [118]	2023
		SILENCE [16]	2023
		LiVo [176]	2024
Recommendation and Selection	Concerns on OSS License Selection	Colazo et al. [177]	2005
		Stewart et al. [178]	2005
		Colazo et al. [179]	2009
		Lindman et al. [180]	2011
		Kashima et al. [181]	2011
		Ghapanchi et al. [61]	2011
		Vendome et al. [167]	2015
		Vendome et al. [182]	2015
		Vendome et al. [168]	2017
		Medappa et al. [183]	2017
		Gamalielsson et al. [184]	2017
	Approaches to Narrow down License Selection	ALP [42]	2019
		findOSSLicense [175]	2019
		LicenseRec [139]	2023
		Zhang et al. [185]	2024

cense types within a project, academic research emphasizes whether specific license obligations are adhered to across different code usage scenarios. In the context of generative code, academic works have extended beyond traditional code tracing to include the analysis of prompts and the code from training datasets. These phenomena highlight the industry's cautious reliance on manual analysis for compliance detection. However, given the increasing complexity of code application scenarios and license terms, more detailed compliance analysis approaches need to be integrated for industry advancements.

C. License Risk Mitigation

In this section, we discuss the countermeasures proposed by academic research, as listed in Table VI.

1) **License Risk Remediation:** To systematically eliminate identified licensing risks, researchers have conducted empirical studies on solutions that have emerged in practice. After investigating solutions to license incompatibilities in 124 OSS packages, German et al. [8] identified twelve categories of remedial strategies, which are applicable to both licensors and licensees, such as loosening license restriction for licensors, modifying code, adjusting licenses, and negotiation with licensors to resolve potential license conflicts. Hammouda et al. [171] proposed the concept of open-source legality patterns in detail, and introduced a number of open-source legality patterns identified in real systems and expert interviews, such as decoupling component interactions, isolating critical business logic, and adjusting licensing schemes, to avoid potential license incompatibility in real-world software architectural design. Scacchi et al. [172] also argued that the coupling relationships between components in open architecture systems can be leveraged to mitigate the risks of license incompatibilities, such as component replacement, architectural refactoring

(e.g., switching from static linking to dynamic linking), and license updates. Mlouki et al. [133] surveyed solutions for resolving conflict between project-declared licenses and code internal licenses across 859 Android applications. They find that developers typically address these issues by changing the licenses or removing the contentious files.

Some researchers also turned these possible operations into automated solutions, to remediate identified legitimate issues as much as they can. This line of research advanced alongside compatibility detection, evolving from conceptual proposals [42], [173], [174] to incompatible license pair-based detection [113], [175], and ultimately to the evaluation of specific license terms [17], [139]. However, this automated approach is limited as there is not always a possible compatible alternative to perfectly resolve the conflicts. In this case, LiResolver [118] was proposed to address this by customizing license terms. LiResolver also seeks licenses that eliminate term conflicts as replacements first. When such a license is unavailable, LiResolver is also the only tool that customizes the terms of an existing license, such as converting the term attitudes of the constraint requirements to the most stringent option or generating exceptions to the license, to resolve conflicts. This paper also mentioned that if the conflicting components are developed by the same developer, replacing the license directly could be considered. Xu et al. [16] further extended license risk remediation by library migration techniques. They first conducted an empirical study on common remediation methods for incompatibilities in the PyPI ecosystem, identifying five popular strategies: *Migration*, *Removal*, *Pin Version*, *Negotiation*, and *Change Own License*. They collected data on migrated packages from their previous work [186] and license information across different package versions, then used an SMT solver to determine if conflict resolution could be achieved by replacing conflicting components. Huang et al. [176] also delved into the issue of missing modification notices which are one of the most common causes of license non-compliance. They proposed LiVo to detect gaps in required notices within GitHub commit logs and supplement missing details to ensure compliance. LiVo is also the only work to date that resolves missing modification declarations.

Industry Perspective: The mitigation of identified license risks is relatively less concerning in industrial SCA tools. Typically, they only either leverage metadata to automatically generate attribution lists for third-party dependencies to address missing attribution issues [62], [67], [72], [88], or simply provide general suggestion for manual remediation in generated risk reports [64], [66], [70], [85], [89], [94], which heavily rely on manual review and operations. However, it is non-trivial for developers to understand the rights and obligations behind legal terms in generated reports, which could largely hinder the mitigation of license-related risks.

Remark 4: Compared to academic research that mostly focuses on the automation of license risk mitigation, industrial tools are much more cautious and careful when introducing automated solutions. However, simply leaving users alone with detailed reports with general suggestions

also does not really solve the challenge of developers who are not familiar with legitimate issues. To this end, the issue of how to reach an agreement between automation and acceptable manual validation should be reconsidered by both academic researchers and industrial vendors.

2) License Recommendation and Selection: Apart from remediating identified legitimate issues, how to select and assign proper licenses to user projects is another widely concerned scenario by developers. To this end, researchers have also delved into the recommendation and selection of licenses to developers.

• **Concerns on OSS License Selection:** The selection of an appropriate open-source license plays a key role in promoting sustainable software development. [179]. Empirical studies investigating the impact of license types on open-source projects have analyzed diverse scenarios, encompassing widely used projects on GitHub [167], [168], [183], large-scale repositories from Debian and SourceForge [177], [181], projects supported by companies [61], [178], [183], and the top-ranked OSS projects on OpenHub [184]. These studies collectively reveal that permissive licenses are particularly favored due to their ability to attract commercial sponsorship, increase developer engagement, and facilitate code reuse across diverse contexts. In contrast, copyleft licenses, while potentially limiting user interest and project vitality, provide distinct advantages such as better safeguarding openness, fostering stronger alignment between developers and their communities, and attracting intrinsically motivated contributors.

Apart from examining how license strictness affects project success, other research has explored additional factors that shape license decision-making. Vendome et al. [182] analyzed 16,221 Java projects and surveyed 138 developers, identifying critical drivers such as compatibility risks, commercial reuse, and user feedback. Lindman et al. [180] analyzed related factors from a commercial perspective and, through case studies and interviews, identified key drivers such as externalities (i.e., compatibility with existing systems), developer motivation, community leadership, and company size.

• **Approaches to Narrow down License Selection:** Some researchers also focus on developing assistant tools to help users narrow down and select appropriate licenses for their own projects. LicenseRec [139] asked users seven questions to obtain their requirements on licenses, then filtered out suitable licenses by matching terms recommended to users by referring to their knowledge base [141].

Apart from user needs, some researchers also investigated the practice of considering similar projects as a basis for license recommendation, in various scenarios. Zhang et al. [185] focused on license selection for software in the electric power industry. Considering that it could be difficult for non-developers to understand the complex concepts and threats of software licenses, they rely on project information (e.g., the project description, readme text, chosen topics, programming language, project size, and the year) to cluster existing power-related GitHub repositories, and recommend software licenses to these users to simplify their license selection.

ALP [42] incorporates the project's historical versions and change information into license selection considerations. It

considers various project features to recommend licenses, including license-related content in code-inline text and changed text, the impact of project historical versions on license selection, the content of project document texts, and the license differences in co-changed files. The study uses the Conditional Random Field (CRF) [187] algorithm to predict licenses based on different features respectively. If there are discrepancies in the predictions, a ranking-learning algorithm provided by libSVM [188] is used to rank the features contributing to the discrepancies, with the highest-ranked instance being used as the basis for the recommended license.

FindOSSLicense [175] refined its license selection approach by integrating both user and project properties into its recommendations. By evaluating 33 well-known licenses, they first identified 38 distinct license terms, and further gathered user requirements through questionnaires, such as user demand on license restrictions. These insights helped filter license terms to generate preliminary recommendations. After that, the tool factored in the programming language and application type of user projects and recommended licenses that align with these properties. Lastly, findOSSLicense utilized a collaborative filtering algorithm that leveraged these properties and user preferences for licenses to make final recommendations.

Industry Perspective: Unlike academic research that has delved into the selection of licenses from various aspects concerned by different stakeholders, industrial SCA tools seldom recommend licenses to user projects. After going through all leading SCA tools in Table II, only JFrog [92] and GitHub [97] specifically give recommendations to repositories that have no license assigned, while their recommendations are also straightforward without additional guidance to assist with the license selection process.

Remark 5: There is an obvious delay in the adaption of license recommendation approaches to the industry. Academic researchers have investigated a lot in the factors that could influence the license selection, while only a few of them are incorporated in existing SCA tools. This also unveils that existing SCA tools are less interested in solving license missing problems in user projects, which should be further enhanced to complete the lifecycle of license management in SCA tools.

V. DISCUSSION

In this section, we discuss the existing challenges in the procedures, and outline potential opportunities for future development in this area. Additionally, we also provide practical recommendations for practitioners to enhance their license management processes.

A. Challenges in Current License Risk Management

Challenge 1: License Proliferation and Lagging Tool Support. With the increasing specified requirements of project management, existing licenses are gradually customized and proliferated to satisfy maintainers' needs. However, such customizations could introduce rights and obligations that are not commonly defined in existing mainstream software licenses,

TABLE VII
NUMBER OF LICENSES IDENTIFIED BY IDENTIFICATION METHODS

Tool or First Author	Database	#License
Wolter et al. [114]	ScanCode LicenseDB	2000+
Nomos [11]	FOSSology Database	1700+
Monk [11]		
Wolter et al. [114]		
Wu et al. [9]	SPDX License List	606
Ninka [10]	Ninka Database	112
Higashi et al. [108]		
Higashi et al. [109]		
ASLA [106]	ASLA Database	37
SORREL [113]	SORREL Database	16
Di Penta et al. [111]	Google Code Search	Inaccessible
LChecker [112]		

leading to the increasing difficulty for existing license management tools to identify, parse, and ensure their compatibility and compliance with other licenses.

It is evident that maintaining a comprehensive and up-to-date license knowledge base is a critical foundation for effective license management methods and tools. However, license proliferation often leads to fragmented, flawed, or outdated knowledge, diminishing the practical utility of these methods and tools. As pointed out by OSI, the extensive growth of OSS licenses (i.e., license proliferation) not only complicates the selection process for licensors but also leads to incompatibilities and confusion over the terms in multi-license distributions [189]. For instance, both rule-based and fuzzy matching methods for license identification necessitate regular updates and meticulous maintenance to remain effective. Their success hinges critically on the completeness and accuracy of the license rule sets employed.

We conducted an in-depth analysis of collected papers to ascertain the scope of license types encompassed by various identification tools, as outlined in these papers and their related tool documentation. As presented in Table VII, existing license identification techniques significantly vary on the scope of software licenses they support. However, the covered licenses are still far from real-world license adoptions. According to Jesus M. et al. [190], 6.9 million unique license declarations are identified from Software Heritage. Although there could be many variations and exceptions of mainstream licenses [15], [118], we believe the scope of important licenses is still far larger than current support. Moreover, the effective license risk analysis also heavily relies on the reliable identification of granted software licenses. For instance, many existing tools rely on well-built compatibility databases to detect license violations, while the most advanced compatibility matrices only cover over 100 licenses [141], [191], which is far from comprehensive coverage for the proliferated licenses and require massive efforts to make up. To this end, with the increasing growth of customized licenses, it could be even more challenging to sustain these knowledge-based solutions for license risk management, and regulations are urgently required to prevent the chaos of license proliferation.

Challenge 2: Missing Fine-Grained Meta-Model of Software Licenses. As a gradual common practice to clarify license restrictions, license terms are increasingly being for-

TABLE VIII
OVERVIEW OF LICENSE IDENTIFICATION METHODS USED IN LICENSE
RISK ASSESSMENT RESEARCH

Tool or First Author	Year	Name Identification						Term Identification
		Manual	Ninka	FOSSology	ScanCode	Metadata	Others	
German et al. [60]	2009			✓				
Di Penta et al. [165]	2009					✓		
LChecker [112]	2010					✓		
German et al. [131]	2010		✓			✓		
Di Penta et al. [166]	2010			✓				
BAT [162]	2011						✓	
Kenen [24]	2012	✓	✓					
Alsbaugh et al. [138]	2012							✓
Mathur et al. [132]	2012					✓		
CBDG [23]	2014		✓					
Wu et al. [18]	2015		✓					
Vendome et al. [167]	2015		✓			✓		
Mlouki et al. [133]	2016		✓					
SPDX-VT [14]	2017	✓		✓				
Wu et al. [19]	2017		✓					
OSSPolice [21]	2017			✓		✓		
Vendome et al. [168]	2017		✓					
An et al. [151]	2017	✓						
Lotter et al. [152]	2018					✓		
Feng et al. [163]	2019	✓					✓	
Ragk. et al. [154]	2019	✓						
Baltes et al. [153]	2019						✓	
OSLDetector [164]	2020					✓		
Golubev et al. [155]	2020		✓			✓		
Moraes et al. [135]	2021			✓				
Serafini et al. [156]	2022						✓	
Higashi et al. [22]	2022		✓					
Sorrel [113]	2021						✓	
Qiu et al. [134]	2021					✓		
Makari et al. [136]	2022					✓		
LiDetector [15]	2023		✓			✓		✓
LiResolver [118]	2023		✓					✓
LicenseRec [139]	2023			✓		✓		
DIKE [17]	2023						✓	✓
SILENCE [16]	2023			✓		✓		
CODEIPROMPT [158]	2023					✓		
Reid et al. [169]	2023					✓		
Harutyunyan et al. [129]	2023						✓	
Liu et al. [43]	2024					✓		✓
Wu et al. [9]	2024						✓	
Antelmi et al. [137]	2024			✓				
Sun et al. [130]	2024					✓		
ModelGo [160]	2024							✓
Total (43)		5	12	4	4	19	9	6

manually modeled and operationalized for settling violations as the supply of license knowledge [15], [118], [192]. However, there are still disputes on the meta-model to guide the understanding of software licenses.

We reviewed the license identification approaches adopted and mentioned in license risk assessment research to inspect the meta-model behind the understanding of software licenses. As presented in Table VIII, apart from manual analysis and package metadata, existing research heavily relies on Ninka, FOSSology, and ScanCode to support the identification of licenses. Specifically, there is a clear shift in the analyzed targets in research on license risk detection. Before 2017, these research works mainly relied on these tools to retrieve information, identify software licenses, and then conduct studies at the license level, while after 2023, researchers turned to detecting the detailed license terms and conducting finer-grained analysis on term-level conflicts. This shifting indicates the need for a finer-grained interpretation of software licenses for follow-up research, as well as real-world requirements.

Current research works based on license terms are gradually sorting out a standardized meta-model for license terms. Existing software license platforms, such as TLDRLegal [146], Choosealicense [145], and OpenEuler [147], are proposed to index popular software licenses with their terms. Based on these term templates, researchers have also increasingly adopted SOTA techniques to extract license terms [193]. However, existing term-based license meta-models still suffer from issues such as low coverage, lack of standardization, and practical limitations. As revealed by Liu et al. [43], there are significant gaps among these license indexing platforms. First, the interpretation of software licenses requires domain knowledge of laws and regulations from experts, which is

TABLE IX
OVERVIEW OF LICENSE OBLIGATIONS DETECTED IN COMPLIANCE
ASSESSMENT RESEARCH

Tool or First Author	Year	Include Copyright	Include License	Include Notice	Disclose Source	Sublicense	Relicense	State Changes
Di Penta et al. [165]	2009	✓						
Molina et al. [161]	2010				✓			
Di Penta et al. [166]	2010						✓	
BAT [162]	2011				✓			
Wu et al. [18]	2015						✓	
Vendome et al. [167]	2015						✓	
Mlouki et al. [133]	2016					✓	✓	
Wu et al. [19]	2017						✓	
OSSPolice [21]	2017				✓			
An et al. [151]	2017	✓	✓	✓			✓	
Vendome et al. [168]	2017						✓	
Lotter et al. [152]	2018		✓	✓			✓	
Baltes et al. [153]	2019			✓			✓	
Ragk. et al. [154]	2019		✓					
Feng et al. [163]	2019				✓			
Golubev et al. [155]	2020						✓	
OSLDetector [164]	2020							
CODEIPROMPT [158]	2023		✓	✓				
Reid et al. [169]	2023	✓	✓	✓				
ModelGo [160]	2024		✓	✓	✓	✓	✓	✓
Kapitsaki et al. [159]	2024			✓				
Total (21)		3	6	6	6	2	11	1

difficult to automate and extend to cover all software licenses. Next, the interpretation model of software licenses, i.e., term definitions, attitude judgment, conflict determination, also vary among different platforms, which could bias the judgment of users and compromise the real-world adoption of software license tools developed from software engineering disciplines. **Challenge 3: Limited Investigation of License Compliance.** Researchers are increasingly focusing on license compliance to mitigate potential legitimate risks. However, compared to the various obligations, existing concerns are mainly on limited scopes. As presented in Table IX, we summarized obligations discussed and emphasized in existing research works on license compliance assessment. Our investigation reveals that most existing research concentrates on obligations of *Relicense*, and some others involve obligations of *Include Copyright*, *Include License*, *Include Notice*, *Disclose Source*, *Sublicense* and *State Changes*. Regarding the terms concluded in previous works [15], [43], there are far more obligations to investigate for compliance analysis. To date, we have not found any research specifically addressing the detection of compliance behaviors related to certain terms in the existing model [15], such as *Include Original*, *Rename*, and *Contact Author*. While the risk levels associated with these terms may be lower than those previously mentioned, they still present potential threats, especially given the lack of adequate techniques to detect these compliance violations.

Challenge 4: Ambiguity Definition in License Context. The definition of software licenses often involves specific terms from the discipline of law, while some terms, from the perspective of measurable metrics, are still ambiguous to detect, complicating how practitioners understand, utilize, and manage their licenses. However, the academic community has yet to engage in a thorough discussion of these issues.

A typical example of ambiguous terms from licenses is the concept of *Derivative Works*. Many licenses stipulate that the distribution or modification of source code or derivative works incur specific obligations. Yet, the evaluation of what constitutes a derivative work often lacks a consistent, quantifiable approach. Existing researches in license risk analysis mostly utilize code clone detection tools (e.g., CCFinderX [194], SourcererCC [195]) to identify derived code from third parties, which usually rely on empirical thresholds, such as length of

similar codes snippets [195] and similarity on features of code semantics (e.g., Abstract Syntax Tree [196], Program Dependency Graph [197]) to approach the behavior of real copies. However, similar code snippets are not a reliable indicator of actual code reuse, and there remains no clear definition of how such similarities relate to *Derivative Works*. Although this term is primarily a legal issue needing a clear definition, these intricacies make automated code derivation determination infeasible, highlighting the need for more robust, objective methodologies in licensing compliance analysis.

Besides the *derivative works*, some other terms are also not captured by existing tools. For instance, there is no comprehensive taxonomy on the approaches and formats of third-party usage. In early licenses, there are no explanations of the approaches and formats of how artifacts can be used. Afterward, software licenses gradually distinguish artifact formats, such as source code, binaries, and executables, as well as how they are integrated, such as package manager dependencies, static and dynamic links [198], web API calls [199], Software Development Kits (SDKs) [200], to grant different rights and obligations. However, current research also only focuses on limited types of third-party artifact reuses, and there is a conspicuous dearth of research that systematically categorizes the myriad software distribution methods specified by licenses and undertakes targeted compliance assessments. The community is also gradually introducing new scenarios of software usage that could challenge existing software license systems, such as the rising concern about whether open-source codes could be used to train CodeLLMs [158], is not scoped by most of the existing popular licenses.

Moreover, some rights and obligations are restricted with specific conditions that are based on abstract, context-dependent, or subjective criteria, making it difficult to interpret the semantics and detect the potential license risks. For example, the BSD 3-Clause No Nuclear License [201] prohibits the use of software in nuclear facilities, while the JSON License [202] includes a moral clause stating that “The software shall be used for good, not evil.” Such terms lack precise definitions, leading to uncertainty in compliance assessment and legal interpretation.

These gaps underscore the need for more structured and well-defined models and explanations for license interpretation, along with standardized regulations for more comprehensive and unambiguous license risk management tools.

Challenge 5: License Tampering During Code Reuse.

With the prevalence of source code reuse in modern software development, license missing or even tampering during user copy-and-paste reuse are gradually common. This often involves modifying or completely omitting the original license information, such as removing or omitting header comments that contain copyright notices, which could largely bias the existing solutions on license risk detection. For instance, a developer might copy a function from a project licensed under GPL and integrate it into a proprietary project without disclosing the source or adhering to the GPL’s terms about open-sourcing the *derivative work*. If this project is reused by downstream users, the lack of declarations may prevent text-based license identification tools from detecting GPL content

TABLE X
LICENSE RISK REMEDIATION STRATEGIES

Tool or First Author	Obligation Complement	License Altering			Component Replacing			
		Relicense	Customization	Negotiation	Code Refactoring	Reuse via API	Components Replacement	Components Removal
Hammouda et al. [171]		•						•
German et al. [8]		•	•	•		•	•	
Scacchi et al. [172]								
Vendome et al. [174]		•						
Mlouki et al. [133]		•						•
Viseur [173]		•						
ALP [42]		•						
findOSSLicense [175]		✓						
Sorrel [113]		✓						
LicenseRec [139]		✓						
DIKE [17]		✓			•			•
LiResolver [118]		✓	✓	•				
SILENCE [16]		✓		•			✓	
LiVo [176]	✓							✓
✓ Strategy was implemented • Strategy was collected or mentioned								

✓: Strategy was implemented. •: Strategy was collected or mentioned.

in third-party dependencies, posing a risk of GPL violations.

The reasons behind such license tampering could vary. Developers could deliberately attempt to avoid the complications of license compliance, or might not understand the implications of mixing code under different licenses, such as the legal requirement to release proprietary software under a GPL if it includes GPL-licensed code. However, legally, this can lead to lawsuits and demands for statutory damages. From a business perspective, companies can suffer severe reputational damage if they are seen as copyright violators, potentially leading to lost business opportunities.

Moreover, after decades of unlimited source code reuse, there could be already plenty of source codes not associated with their original licenses. Especially, the wide adoption of LLM-based code generation in modern software development could even aggravate the chaos of license tampering. Considering most existing license risk management tools, it is daunting and challenging to properly manage and govern the ecosystem-wide license risks in this context by existing techniques. To this end, comprehensive identification and source tracing of source code reuse is urgently required to clear out the original licenses, especially for those commonly reused code snippets.

Challenge 6: Limited Practicability of License Risk Remediation. Considering the complexity and prevalence of license risks, researchers from both academia and the industry have proposed various solutions to assist and automate the mitigation of potential legitimate issues. Apart from satisfying required conditions, e.g., making up necessary notices and statements [176], typically, existing solutions can be classified into two major types, 1) altering licenses of user projects, such as relicense, license customization, license exception, and negotiation, and 2) replacing third-party components, such as code refactoring, reuse via APIs, component replacement, and component removal, to avoid potential license incompatibility or non-compliance, as presented in Table X. However, these mitigation strategies are still limited by their practicability.

For license-altering solutions, although there are hundreds of mainstream OSS licenses, there is not always a suitable one to handle all potential legitimate risks. Next, replacing licenses could be challenging to accomplish. An open-source

TABLE XI
LICENSE RECOMMENDATION AND SELECTION FACTORS

Tool or First Author	Commercial Impact	Community Engagement	Compatibility	Project Nature	Developers' Requirement
Colazo et al. [177]	•	•	•		
Stewart et al. [178]	•	•			
Colazo et al. [179]	•	•			
Lindman et al. [180]		•	•		
Kashima et al. [181]	•	•	•	•	
Ghapanchi et al. [61]		•			
Vendome et al. [167]	•	•	•		
Vendome et al. [182]	•	•	•		
Vendome et al. [168]	•	•	•		
Medappa et al. [183]	•				•
Gamalielsson et al. [184]		•	•		
ALP [42]			✓	✓	
findOSSLicense [175]			✓	✓	✓
LicenseRec [139]			✓		✓
Zhang et al. [185]				✓	

✓ : Factor was used for implementation. • : Factor was mentioned.

project often involves many contributors, while the license replacement should be granted by all contributors, which could be difficult. A typical example is the multi-year efforts of relicensing agreement collection of LLVM [203] to provide patent protection for LLVM users. As for license customization and exceptions, although these solutions could temporarily avoid potential legitimate risks, they could exacerbate the license proliferation in real-world applications, which may lead to a more severe situation for license identification and governance. Moreover, since licenses are usually written in professional languages, freely modifying license text or generating licenses could face potential threats to the rigorously and legality of generated text. As for negotiation with owners of conflicted components proposed by existing works [8], [118], it is a case-by-case solution and is still not systematically studied yet.

For third-party component replacement solutions, the primary concern is their impact on the functionality of projects. It's relatively rare for licenses to change during software version updates, thus a common approach is to replace the third-party artifacts with restrictive licenses to those under more permissive ones. However, suitable alternative libraries that meet specific user requirements are not always available. Even when alternatives exist, automating the migration of libraries remains a significant challenge. With the rise of generative software engineering, there is potential for large language models (LLMs) to facilitate this process. Nonetheless, the use of LLMs introduces its own complexities, particularly concerning the copyright status of code generated by LLMs. This indicates that while LLMs could streamline library migration, there remains a considerable path ahead in fully realizing their potential and addressing associated legal concerns.

Challenge 7: Inadequate Requirement Mining for License Recommendation. Besides remediating license risks, license recommendation techniques also play important roles in avoiding potential legitimate risks. Existing works have identified various factors influencing license selection and have developed recommendation tools to assist developers, as identified in Table XI, while these existing tools fail to integrate these considerations in subtle ways.

For instance, one of the primary concerned factors involves compatibility checks, akin to those used in license risk remediation. This approach, however, struggles with a limited availability of compatible licenses, often leading to recommen-

dations that do not fully meet the specific needs of different projects. As a result, developers and organizations are increasingly turning towards customized licenses, resulting in license proliferation, which could create a fragmented landscape of licensing options that may not always adhere to best practices for ecosystem-wide license management and standardization. Apart from this, requirements from developers and project natures are also considered by existing methods. However, the gathering of developer requirements is confined largely to the terms specified within licenses, failing to capture the broader, more detailed requirements of the application scenarios for which the software is intended, such as moral reasons and community influence [182]. These tools also fail to account for the complexities of *commercial impact* and *community engagement*, such as whether a recommended license aligns with corporate preferences to attract sponsorship, guarantees openness, or fosters community involvement in development and maintenance [177], [178], [183].

Furthermore, while research has outlined numerous factors affecting license selection, a systematic discussion is still lacking. Existing research has yet to establish a comprehensive framework that fully summarizes and organizes all considerations. There is no clear consensus on how to weigh or balance different factors. As a result, current approaches remain fragmented, limiting their effectiveness in providing well-rounded recommendations for developers in diverse contexts.

B. Opportunities for Future Research

Considering the shared trends among the challenges currently faced in License Risk Management and the progression of existing licenses, we identify potential opportunities for future advancements in this research area.

Opportunity 1: Standardized License Meta-Model and Framework. As revealed by the SLR, the lack of standardization becomes increasingly problematic as the proliferation of licenses grows, leading to confusion and complexity for management. As user requirements evolve and software reuse scenarios increase in complexity, the need for standardized definitions and terms becomes even more critical. Standardizing these elements would improve understanding of the licenses and enhance risk management associated with them. There is a promising potential for experts to endorse the relationships between existing licenses and these standardized definitional entities, progressively advocating for the incorporation of more granular license content as the standard for future licensing practices.

However, existing license knowledge bases mainly focus on prevalent licenses, presenting significant deficiencies and inadequacies for the modeling of license semantics, especially for conditions and restrictions raised from the emerging requirements of developers, such as the evolving ways of reuse, new types of obligations, and exceptions for corner cases, etc. For example, to address the limitations of traditional licenses in expressing new ways of reusing open source LLMs, such as leveraging a model's weights, parameters, activations, and other related materials for methods like fine-tuning, evaluation, or distillation to improve other models, many large model

providers carefully design customized licenses during open-source releases to avoid ambiguity [204], [205].

To this end, a comprehensive analysis and standardization of software licenses could not only be useful for governance but also inspire the flexibility to satisfy the demands of real-world license applications efficiently. However, constructing such a meta-model is a highly challenging task that requires substantial collaborative efforts from the OSS community, industry, and academia, especially from the legal field to ensure professional and precise interpretations of license semantics and risks. Quantifying such details will support more accurate license management and help reduce license content disputes.

Opportunity 2: Measurable Metrics for Automated License Violation Detection. Current research on license risk detection in software management has predominantly focused on the more observable aspects such as the inclusion of licenses, copyrights, notices, and conflicts arising from contradictory licensing terms. However, since these compliance requirements cannot be effectively measured, automated detection is infeasible, making manual review indispensable. Furthermore, many obligations within the existing license term model remain undetectable in compliance. For example, proper use of trademarks, which requires software to avoid misleading representations that could confuse the trademark with a generic term, is rarely monitored effectively. The adherence to patent stipulations and handling of promotional texts, such as avoiding unauthorized endorsements or implied associations, are also critical yet difficult to quantify. The development of measurable metrics, to gauge these less tangible aspects of licensing could revolutionize the approaches to managing and enforcing software licenses.

Another complex area involves the legal interpretation of terms used in software licenses, which are often steeped in domain-specific jargon that does not easily lend itself to quantifiable metrics. For instance, the term *derivative works* is frequently used in licenses but lacks a clear, universally accepted definition. This ambiguity poses a challenge that determining whether a piece of software that shares similarities with another is simply a coincidence or actually constitutes a *derivative work* can be highly subjective and contentious. Additionally, the scope of what is considered source code similarity can vary, further complicating efforts to automate the detection of such violations. These issues highlight the necessity for more sophisticated tools that can interpret and measure these complex legal concepts within the framework of software license management. Moreover, the interpretation of these domain-specific terms is highly related to local proprietary and copyright laws, different justice systems could result in inconsistent outcomes in similar legal cases, and more investigations on lawsuit cases and corporations with legal professions are expected to strengthen the alignment of software license risk management.

To address these complexities, there is a pressing need for closer collaboration between the academic field of software management and legal experts specializing in intellectual property and software licensing. By fostering partnerships between these disciplines, it is possible to bridge the gap between legal theory and practical, measurable enforcement

in software license management. Such interdisciplinary efforts are crucial for advancing research in license violation detection and ensuring that software management evolves to meet the changing landscapes of law and technology.

Opportunity 3: License Management of Contributors. Developing a single OSS project frequently involves numerous contributors, who hold the copyrights to their contributions. However, these contributors could potentially have various views on the roadmap, and management principles of OSS projects, which could lead to divergences. Historical instances, such as the fork of XFree86 into X.Org Server due to license disagreements [206], and the creation of LibreOffice in response to dissatisfaction with Oracle’s management of the OpenOffice.org suite [207], exemplify the impact of such divergences. To mitigate disputes and streamline management, agreements such as the Contributor License Agreement (CLA) and the Developer Certificate of Origin (DCO) have been instituted [208], [209]. These agreements are intended to ensure that contributors retain copyright over their code and agree to transfer these rights to the project maintainers.

Nonetheless, according to our review, we have found no papers focusing on the implementation and impact of CLAs and DCOs. There are broad directions for future research works to address the critical problem, regarding the prevalence of these agreements in OSS projects. For instance, the effectiveness of CLAs and DCOs in protecting intellectual property rights, the administrative burdens they impose on contributors and maintainers, such as standardized understanding, violation risks, mitigation strategies, and their role in fostering or hindering collaboration within OSS communities. Additionally, the legal implications of these agreements in different jurisdictions and their compatibility with various licensing models represent significant challenges that require detailed examination. To address these issues, future academic endeavors could benefit from collaborations with open-source repositories and developers to investigate the current utilization and evolution of DCOs and CLAs.

Opportunity 4: License Risks for Generative Software Engineering. The prevalence of generative software engineering based on LLMs has seen a significant rise in recent years. These models, such as CodeGen [210], Copilot [211], and ChatGPT [212], have demonstrated remarkable capabilities in generating code, automating repetitive tasks, and even assisting in software design and debugging. Developers increasingly rely on these models to streamline workflows and achieve higher efficiency. However, the integration of LLMs in software engineering introduces potential threats and challenges related to license management. One significant concern is the inadvertent incorporation of licensed code snippets generated by LLMs, which could lead to intellectual property violations [6], [213]. Given the vast training data these models are exposed to, it is challenging to ensure that the generated code complies with open-source licensing requirements, since it is still a gray area to determine whether similar generated code constitutes a derivative of copyrighted work [214]. Additionally, the dynamic nature of LLMs, which continuously learn and adapt, complicates the tracking of license compliance over time. The opacity of these model training datasets further

exacerbates this issue [158], making it difficult to verify the provenance and licensing status of the generated code.

To address these challenges, research should be further conducted for license management in the context of generative software engineering. One promising area is the license compliance check for code generated by LLMs and ensuring it adheres to relevant rights and obligations by its original licenses. Additionally, there is a need for transparency in the training datasets of LLMs, allowing for better auditability and verification of code origins. Research could also explore the development of standardized frameworks for integrating license management into the process [159] of adoption practices of generated code by LLMs, ensuring that licensing considerations are embedded from the outset. Moreover, another critical issue that may be beyond the scope of OSS license management is that there are urgent needs for a more robust framework of legal explanations of granted rights and obligations in the generative context, such as the boundary of derivative work, rights of use in LLM-specific scenarios, etc. However, this may require further dedication from academia, industry, open-source communities, and laws to work together and derive best practices and guidelines to navigate the complexities of license management in the era of generative software engineering.

C. Recommendation for Practitioners

In this section, we offer actionable advice for different stakeholders, based on the insights gained from our study.

Recommendations for OSS Developers:

1) Effective license management is crucial for OSS developers to ensure legal compliance and foster community trust. As stipulated in existing standards, e.g., ISO/IEC 5230 [29], software license compliance should be rigorously restricted in the software supply chain throughout software lifecycles. We encourage OSS developers to maintain explicit and up-to-date licensing information in their repositories, i.e., a clear LICENSE file at the root of the repository.

2) When selecting software licenses, the considerations outlined in Table XI provide a useful starting point for decision-making. However, existing tools based on these factors may not account for all relevant concerns or usage scenarios. Therefore, developers should still assess additional factors according to the specific context and goals of their projects.

3) Regularly auditing third-party dependencies to verify that their licenses are compatible with the chosen license, by utilizing SCA tools, is also highly recommended. Specifically, it is important to carefully keep an eye on the obligations and conditions to fulfill before integration, and tools at term-level [15], [17], [176] and existing license interpretation websites [145], [146], can be helpful.

4) OSS developers could receive massive commits or PR to their repositories, or they contribute to others' repositories, it is also important to clearly document and review how these contributions are licensed, preferably through a Contributor License Agreement (CLA) or Developer Certificate of Origin (DCO), to avoid future legal complexities.

5) Developers should also stay informed about licensing trends and best practices to address any emerging risks or

conflicts proactively, helping to safeguard your project's legal integrity and reputation (e.g., by following initiatives and discussion of OSI and the OpenChain Project [215]).

Recommendations for Users:

1) For users of OSS, understanding license management is key to ensuring proper and legal use of the software. Users should always review the license associated with an OSS project before reuse, especially in commercial or proprietary environments, to understand any obligations or restrictions.

2) Prevalent licenses like MIT or Apache 2.0 are permissive, allowing broad use with minimal requirements, while copyleft licenses like GPL impose stricter conditions, such as sharing derivative works under the same license. Therefore, it is also important to pay attention to clauses about attribution, redistribution, and modifications to ensure compliance.

3) As revealed by existing research [176], less concerned obligations, such as placing appropriate notices, are also easy to overlook. Users should examine each license obligation systematically based on the existing license term framework [43]. When faced with ambiguous definitions, legal professionals should be consulted to ensure accurate understanding.

4) If OSS is integrated into users' own projects, they should carefully verify that the licenses are compatible with the intended use and distribution model.

5) When using OSS in production environments, SCA tools are highly encouraged to integrate to help track and manage licenses, especially the rights granted on different integration ways [27], to ensure adherence to all legal requirements and maintain proper documentation.

Recommendations for Third-Party Auditors:

1) For third-party auditors evaluating OSS projects, a more comprehensive approach to license management is essential. As reviewed in existing research [122], auditors could suffer from the limited scope of licenses. They should not only focus on popular licenses like MIT and GPL, but also include more niche licenses to address existing license proliferation.

2) Accurate license identification should also extend beyond the LICENSE file to include code comments, sub-folders, and documentation, uncovering hidden or conflicting licenses. Based on these, incompatibility and noncompliance detection should consider both license types and the context of usage, like dynamic versus static linking, for precise risk assessment.

3) Term-based checks are also crucial, apart from specific license terms such as redistribution and export rights that are directly related to commercial activities, obligations that are easily overlooked, such as including necessary notices and copyright information, should also be included by auditors.

4) More practical remediation strategies are highly encouraged to be implemented, such as replacing incompatible dependencies or isolating problematic components, these could significantly help developers and users with flexible paths to achieve compliance without sacrificing functionality.

D. Limitations and Threats to Validity

We also discuss the potential threats to the validity and limitations of this SLR as follows.

- **Construct Validity.** One potential threat arises from the ambiguity and inconsistency of definitions and terminology

across existing literature, such as *conflict* and *compatibility*. To address this issue, we clarified key concepts in Section II. Building on this foundation, we developed a unified taxonomy to ensure consistent and systematic classification and synthesis of the literature. In addition, the manual processes in this study could be subjective. First, some screening criteria (ie, EC3 and EC6) were applied entirely based on the author’s judgment. Second, the classification of papers also involved potential bias. To mitigate subjectivity, three authors independently conducted manual analyses, cross-validated their results, and resolved any discrepancies through consensus discussions. To further enhance the reliability, we calculated the average Cohen’s κ scores for both tasks (i.e., 0.95 and 0.87), indicating a high level of agreement.

• **Internal Validity.** Including low-quality or irrelevant literature may threaten the internal validity. We carefully constructed search keywords and queried five prominent electronic databases (IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, and DBLP) to ensure broad coverage of relevant papers. We further established criteria (i.e., EC3 and EC6) to confirm the relevance of selected papers. In addition, to avoid misleading and unsupported claims, it is necessary to exclude low-quality papers. However, this approach may inadvertently exclude some high-quality work. To address this, we retain only papers published in venues listed in the latest CORE ranking, which is widely recognized as a reliable indicator of venue quality. Since the dataset we adopted may also miss high-quality papers, we employed an iterative snowballing process to include potentially overlooked works.

• **External Validity.** Because there is no established taxonomy for license management, it’s hard to connect academic research to real-world practice. We therefore propose a taxonomy based on the workflow of SCA tools, since these tools are most related to legitimate issues of OSS in the industry. Moreover, as no comprehensive list of SCA tools exists, we followed a widely recognized enterprise report [38] that summarizes the leading SCA tools to select representative study objectives.

VI. CONCLUSION

In this paper, we conducted an SLR on 80 OSS license management-related papers, categorizing the research into three key areas based on industrial practices: license identification, risk detection, and risk mitigation. Based on these, we conducted a comprehensive review to summarize the current state of research, highlight the differences with industry practices, and discuss the challenges, opportunities, and recommendations for different stakeholders. We hope our review and in-depth discussion can assist both academic researchers and industry practitioners in enhancing the governance of legitimate software risks, thereby improving practices within the software engineering community.

ACKNOWLEDGMENTS

This work was supported by the National Key Research and Development Project (No. SQ2024YFE0201727). It is also supported by the National Research Foundation, Singapore, and DSO National Laboratories under the AI Singapore Programme (AISG Award No: AISG2-GC-2023-008); by the National Research Foundation Singapore and the Cyber Security

Agency under the National Cybersecurity R&D Programme (NCRP25-P04-TAICeN). Any opinions, findings and conclusions, or recommendations expressed in these materials are those of the author(s) and do not reflect the views of National Research Foundation, Singapore, Cyber Security Agency of Singapore, Singapore.

REFERENCES

- [1] “A Summary of Census II: Open Source Software Application Libraries the World Depends On - Linux Foundation,” <https://www.linuxfoundation.org/blog/blog/a-summary-of-census-ii-open-source-software-application-libraries-the-world-depends-on>, [Accessed: May 9, 2025].
- [2] C. V. Lopes, P. Maj, P. Martins, V. Saini, D. Yang, J. Zitny, H. Sajani, and J. Vitek, “Déjàvu: a map of code duplicates on github,” *ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–28, 2017.
- [3] Y. Huang, F. Xu, H. Zhou, X. Chen, X. Zhou, and T. Wang, “Towards exploring the code reuse from stack overflow during software development,” in *30th IEEE/ACM International Conference on Program Comprehension*, 2022, pp. 548–559.
- [4] “GOOGLE LLC v. ORACLE AMERICA, INC.” https://www.supremecourt.gov/opinions/20pdf/18-956_d18f.pdf, 2020, [Accessed: May 9, 2025].
- [5] F. E. Team, “Does TikTok Live Studio Violate GPL v2?” <https://fossa.com/blog/does-tiktok-live-studio-violate-the-gpl-v2/>, 2021, [Accessed: May 9, 2025].
- [6] M. Butterick, “We’ve filed a lawsuit challenging github copilot, an ai product that relies on unprecedented open-source software piracy,” <https://githubcopilotlitigation.com>, 2022, [Accessed: May 9, 2025].
- [7] J. Saveri, C. Zirpoli, T. Manfredi, and M. Butterick, “J. doe 1 & j. doe 2 v. github, inc., microsoft corp., openai, inc., et al.” https://githubcopilotlitigation.com/pdf/06823/1-0-github_complaint.pdf, 2022, [Accessed: May 9, 2025].
- [8] D. M. German and A. E. Hassan, “License integration patterns: Addressing license mismatches in component-based development,” in *2009 IEEE 31st international conference on software engineering*. IEEE, 2009, pp. 188–198.
- [9] J. Wu, L. Bao, X. Yang, X. Xia, and X. Hu, “A large-scale empirical study of open source license usage: Practices and challenges,” in *2024 IEEE/ACM 21st International Conference on Mining Software Repositories*. IEEE, 2024, pp. 595–606.
- [10] D. M. German, Y. Manabe, and K. Inoue, “A sentence-matching method for automatic license identification of source code files,” in *25th IEEE/ACM International Conference on Automated Software Engineering*, 2010, pp. 437–446.
- [11] R. Gobeille, “The fossology project,” in *2008 international working conference on Mining software repositories*, 2008, pp. 47–50.
- [12] G. M. Kapitsaki, N. D. Tselikas, and I. E. Foukarakis, “An insight into license tools for open source software systems,” *Journal of Systems and Software*, vol. 102, pp. 72–87, 2015.
- [13] T. Tuunanen, “Tool support for open source software license compliance: The first two decades of the millennium,” *JYU dissertations*, 2021.
- [14] G. M. Kapitsaki, F. Kramer, and N. D. Tselikas, “Automating the license compatibility process in open source software with spdx,” *Journal of systems and software*, vol. 131, pp. 386–401, 2017.
- [15] S. Xu, Y. Gao, L. Fan, Z. Liu, Y. Liu, and H. Ji, “Lidetect: License incompatibility detection for open source software,” *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 1, pp. 1–28, 2023.
- [16] W. Xu, H. He, K. Gao, and M. Zhou, “Understanding and remediating open-source license incompatibilities in the pypi ecosystem,” in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023, pp. 178–190.
- [17] X. Cui, J. Wu, Y. Wu, X. Wang, T. Luo, S. Qu, X. Ling, and M. Yang, “An empirical study of license conflict in free and open source software,” in *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2023, pp. 495–505.
- [18] Y. Wu, Y. Manabe, T. Kanda, D. M. German, and K. Inoue, “A method to detect license inconsistencies in large-scale open source projects,” in *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*. IEEE, 2015, pp. 324–333.

- [19] Wu, Yuhao and Manabe, Yuki and Kanda, Tetsuya and German, Daniel M and Inoue, Katsuro, "Analysis of license inconsistency in large collections of open source projects," *Empirical Software Engineering*, vol. 22, pp. 1194–1222, 2017.
- [20] G. M. Kapitsaki and F. Kramer, "Open source license violation check for spdx files," in *Software Reuse for Dynamic Systems in the Cloud and Beyond: 14th International Conference on Software Reuse, ICSR 2015, Miami, FL, USA, January 4-6, 2015. Proceedings 14*. Springer, 2014, pp. 90–105.
- [21] R. Duan, A. Bijlani, M. Xu, T. Kim, and W. Lee, "Identifying open-source license violation and 1-day security risk at large scale," in *2017 ACM SIGSAC Conference on computer and communications security*, 2017, pp. 2169–2185.
- [22] Y. Higashi, K. Fukui, K. Yutaro, and M. Ohira, "A preliminary analysis of gpl-related license violations in docker images," in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2022, pp. 444–448.
- [23] S. Van Der Burg, E. Dolstra, S. McIntosh, J. Davies, D. M. German, and A. Hemel, "Tracing software build processes to uncover license compliance inconsistencies," in *29th ACM/IEEE international conference on Automated software engineering*, 2014, pp. 731–742.
- [24] D. German and M. Di Penta, "A method for open source license compliance of java applications," *IEEE software*, vol. 29, no. 3, pp. 58–63, 2012.
- [25] D. A. Almeida, G. C. Murphy, G. Wilson, and M. Hoyer, "Do software developers understand open source licenses?" in *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)*. IEEE, 2017, pp. 1–11.
- [26] D. A. Almeida, G. C. Murphy, G. Wilson, and M. Hoyer, "Investigating whether and how software developers understand open source software licensing," *Empirical Software Engineering*, vol. 24, pp. 211–239, 2019.
- [27] N. Wintersgill, T. Stalnaker, L. A. Heymann, O. Chaparro, and D. Poshvanyk, "the law doesn't work like a computer: Exploring software licensing issues faced by legal practitioners," *ACM on Software Engineering*, vol. 1, no. FSE, pp. 882–905, 2024.
- [28] S. Coughlan, "Standardizing open source license compliance with openchain," *Computer*, vol. 53, no. 11, pp. 70–74, 2020.
- [29] International Organization for Standardization and International Electrotechnical Commission, *ISO/IEC 5230:2020 - OpenChain Specification*, <https://www.iso.org/standard/81039.html>, ISO/IEC Std., 2020, [Accessed: May 9, 2025].
- [30] K. Stewart, P. Odenice, and E. Rockett, "Software package data exchange (spdx) specification," *IFOSS L. Rev.*, vol. 2, p. 191, 2010.
- [31] N. Telecommunications and I. Administration, "Software bill of materials," <https://www.ntia.gov/page/software-bill-materials>, [Accessed: May 9, 2025].
- [32] "Software Supply Chain Management — Sonatype," <https://www.sonatype.com/>, 2024, [Accessed: May 9, 2025].
- [33] "Application Security Software(AppSec) — Black Duck," <https://www.blackduck.com/>, 2024, [Accessed: May 9, 2025].
- [34] "Mend.io," <https://www.mend.io/>, 2025, [Accessed: May 9, 2025].
- [35] "Software license management: What, why, how?" <https://cdn.openlm.com/wp-content/uploads/2023/01/Whitepaper-Software-License-Management-What-Why-How-By-OpenLM-2.pdf>, 2022, [Accessed: May 9, 2025].
- [36] "Open-source license - Wikipedia," https://en.wikipedia.org/wiki/Open-source_license, Nov. 2024, [Accessed: May 9, 2025].
- [37] "License proliferation - Wikipedia," https://en.wikipedia.org/wiki/License_proliferation, Nov. 2023, [Accessed: May 9, 2025].
- [38] Synopsys, "2024 Open Source Security and Risk Analysis (OSSRA) Report," <https://www.blackduck.com/content/dam/black-duck/en-us/reports/rep-ossra-2024.pdf>, 2024, accessed on 11/26/2024.
- [39] "License compatibility - Wikipedia," https://en.wikipedia.org/wiki/License_compatibility, [Accessed: May 9, 2025].
- [40] F. Filipsson, "LICENSE COMPLIANCE – A COMPLETE GUIDE," <https://redresscompliance.com/license-compliance-a-complete-guide/>, Jan. 2024, [Accessed: May 9, 2025].
- [41] G. Gangadharan, V. D'Andrea, S. De Paoli, and M. Weiss, "Managing license compliance in free and open source software development," *Information Systems Frontiers*, vol. 14, pp. 143–154, 2012.
- [42] X. Liu, L. Huang, J. Ge, and V. Ng, "Predicting licenses for changed source code," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 686–697.
- [43] T. Liu, C. Liu, T. Liu, H. Wang, G. Wu, Y. Liu, and Y. Zhang, "Catch the butterfly: Peeking into the terms and conflicts among spdx licenses," in *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2024.
- [44] "License Exceptions - Software Package Data Exchange," <https://spdx.org/licenses/exceptions-index.html>, [Accessed: May 9, 2025].
- [45] "GPL linking exception - Wikipedia," https://en.wikipedia.org/wiki/GPL_linking_exception, 2025, [Accessed: May 9, 2025].
- [46] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Technical report, ver. 2.3 ebse technical report. ebse, Tech. Rep., 2007.
- [47] C. Wohlin, "Guidelines for snowballing in systematic literature studies and a replication in software engineering," in *18th international conference on evaluation and assessment in software engineering*, 2014, pp. 1–10.
- [48] L. Li, T. F. Bissyandé, M. Papadakis, S. Rasthofer, A. Bartel, D. Ocleau, J. Klein, and L. Traon, "Static analysis of android apps: A systematic literature review," *Information and Software Technology*, vol. 88, pp. 67–95, 2017.
- [49] X. Zhan, T. Liu, L. Fan, L. Li, S. Chen, X. Luo, and Y. Liu, "Research on third-party libraries in android apps: A taxonomy and systematic literature review," *IEEE Transactions on Software Engineering*, 2021.
- [50] N. R. Haddaway, M. J. Page, C. C. Pritchard, and L. A. McGuinness, "Prisma2020: An r package and shiny app for producing prisma 2020-compliant flow diagrams, with interactivity for optimised digital transparency and open synthesis," *Campbell systematic reviews*, vol. 18, no. 2, p. e1230, 2022.
- [51] "ACM Digital Library," <https://dl.acm.org/>, [Accessed: May 9, 2025].
- [52] "IEEE Xplore Digital Library," <https://ieeexplore.ieee.org/Xplore/home.jsp>, [Accessed: May 9, 2025].
- [53] "SpringerLink," <https://link.springer.com/>, [Accessed: May 9, 2025].
- [54] "ScienceDirect," <https://www.sciencedirect.com/>, [Accessed: May 9, 2025].
- [55] "dblp," <https://dblp.org/>, [Accessed: May 9, 2025].
- [56] "Google Scholar," <https://scholar.google.com/>, [Accessed: May 9, 2025].
- [57] R. Croft, Y. Xie, and M. A. Babar, "Data preparation for software vulnerability prediction: A systematic literature review," *IEEE Transactions on Software Engineering*, vol. 49, no. 3, pp. 1044–1063, 2022.
- [58] "ICORE Conference Portal," <https://portal.core.edu.au/conf-ranks/>, [Accessed: May 9, 2025].
- [59] "CORE Journal Ranking Portal," <https://portal.core.edu.au/jnl-ranks/>, [Accessed: May 9, 2025].
- [60] D. M. German, M. Di Penta, Y.-G. Gueheneuc, and G. Antoniol, "Code siblings: Technical and legal implications of copying code between applications," in *2009 6th IEEE International Working Conference on Mining Software Repositories*. IEEE, 2009, pp. 81–90.
- [61] A. H. Ghapanchi and A. Aurum, "The impact of project licence and operating system on the effectiveness of the defect-fixing process in open source software projects," *International Journal of Business Information Systems*, vol. 8, no. 4, pp. 413–424, 2011.
- [62] "Open Source License Compliance Software For Eradicating Risks," <https://www.sonatype.com/solutions/legal-open-source-license-compliance>, 2024, [Accessed: May 9, 2025].
- [63] "Government intervention, the rise of the SBOM and the evolution of software supply chain security," https://www.sonatype.com/hubfs/White_Papers/Rise_of_SBOM_and_evolution_of_SSC_security.pdf, 2023, [Accessed: May 9, 2025].
- [64] "License Policy Governance," <https://help.sonatype.com/en/license-policy-governance.html>, [Accessed: May 9, 2025].
- [65] "Black Duck Software Composition Analysis," <https://www.blackduck.com/software-composition-analysis-tools/black-duck-sca.html>, 2025, [Accessed: May 9, 2025].
- [66] "Open Source License Compliance," <https://www.blackduck.com/solutions/open-source-security.html>, 2024, [Accessed: May 9, 2025].
- [67] "Managing open source licenses," https://documentation.blackduck.com/bundle/bd-hub/page/Licenses/Overview_LicenseProcess.html, 2025, [Accessed: May 9, 2025].
- [68] "Black Duck: Managing Deep License Data," <https://community.blackduck.com/s/article/Black-Duck-Managing-Deep-License-Data>, 2025, [Accessed: May 9, 2025].
- [69] "Managing open source licenses," <https://documentation.blackduck.com/bundle/bd-hub/page/Licenses/ConflictsAbout.html>, 2025, [Accessed: May 9, 2025].
- [70] "Open source license compliance," <https://www.mend.io/open-source-license-compliance/>, 2025, [Accessed: May 9, 2025].
- [71] "The License Compatibility Report," <https://docs.mend.io/legacy-sca/latest/the-license-compatibility-report>, 2025, [Accessed: May 9, 2025].

- [72] "License attribution for modified transitive dependencies," <https://docs.mend.io/wsk/license-attribution-for-modified-transitive-depend>, 2025, [Accessed: May 9, 2025].
- [73] "Understanding Risk Score Attribution and License Analysis," <https://docs.mend.io/platform/latest/understanding-risk-score-attribution-and-license-a>, 2025, [Accessed: May 9, 2025].
- [74] "Creating Policies for Mend Repository License Checks," <https://docs.mend.io/wsk/creating-policies-for-mend-repository-license-check>, 2025, [Accessed: May 9, 2025].
- [75] "Licenses and Libraries API," <https://docs.mend.io/legacy-sca/latest/licenses-and-libraries-api>, 2025, [Accessed: May 9, 2025].
- [76] "Application Security for the AI Era," <https://www.veracode.com/>, 2025, [Accessed: May 9, 2025].
- [77] "Custom rules for agent-based scanning," https://docs.veracode.com/r/c_sc_policies_custom, 2025, [Accessed: May 9, 2025].
- [78] "Mitigate license risk with Veracode SCA," https://docs.veracode.com/r/Mitigate_License_Risk_with_Veracode_SCA, 2025, [Accessed: May 9, 2025].
- [79] "Snyk," <https://snyk.io/>, 2025, [Accessed: May 9, 2025].
- [80] "Open Source License Compliance Management — Snyk," <https://snyk.io/product/open-source-security-management/license-compliance/>, 2025, [Accessed: May 9, 2025].
- [81] "Snyk License Compliance Management," <https://docs.snyk.io/scan-with-snyk/snyk-open-source/scan-open-source-libraries-and-licenses/snyk-license-compliance-management>, 2025, [Accessed: May 9, 2025].
- [82] "Checkmarx," <https://checkmarx.com/>, 2025, [Accessed: May 9, 2025].
- [83] "Generating a CxOSA Scan Results Report - License Risk and Compliance," https://docs.checkmarx.com/en/34965-46899-generating-a-cxosa-scan-results-report.html#UUID-7616620b-e23c-bd31-f233-89a93a1b5c19_id_GeneratingCxOSAResultsReport-LicenseRiskandCompliance, 2025, [Accessed: May 9, 2025].
- [84] "Everything You Need to Mitigate Open Source Risk," <https://checkmarx.com/cxsca-open-source-scanning/>, 2025, [Accessed: May 9, 2025].
- [85] "Triaging SCA Results," <https://docs.checkmarx.com/en/34965-24923-triaging-sca-results.html#UUID-945497b2-9333-f98f-72d8-21a8a9c31f6a>, 2025, [Accessed: May 9, 2025].
- [86] "Revenera," <https://www.revenera.com/>, 2025, [Accessed: May 9, 2025].
- [87] "Continuous Open Source Software License Compliance," <https://www.revenera.com/software-composition-analysis/business-solutions/open-source-license-compliance>, 2025, [Accessed: May 9, 2025].
- [88] "Automate Attribution Fulfillment via Third-Party Notices Generation," <https://www.revenera.com/sites/default/files/fnci-oss-third-party-notices.pdf>, 2022, [Accessed: May 9, 2025].
- [89] "Technical Due Diligence for Mergers & Acquisitions (M&A)," <https://www.revenera.com/software-composition-analysis/audits-and-services/m-a-support>, 2024, [Accessed: May 9, 2025].
- [90] E. Yaakov, "JFrog," <https://jfrog.com/>, 2024, [Accessed: May 9, 2025].
- [91] E. Yaakov, "Get Your License Compliance Reports with a Click of a Button," <https://jfrog.com/blog/get-your-license-compliance-reports-with-a-click-of-a-button/>, 2018, [Accessed: May 9, 2025].
- [92] "Managing Compliance Licenses," <https://jfrog.com/help/r/jfrog-security-user-guide/products/xray/features-and-capabilities/sca/legal>, 2024, (Accessed on 11/11/2024).
- [93] "Palo Alto Networks," <https://www.paloaltonetworks.com/>, 2025, [Accessed: May 9, 2025].
- [94] "License Compliance in Software Composition Analysis (SCA)," <https://docs.prismacloud.io/en/enterprise-edition/content-collections/application-security/risk-management/monitor-and-manage-code-build/software-composition-analysis/license-compliance-in-sca>, 2024, [Accessed: May 9, 2025].
- [95] "GitHub," <https://github.com/>, 2025, [Accessed: May 9, 2025].
- [96] "Dependabot quickstart guide - GitHub Docs," <https://docs.github.com/en/code-security/getting-started/dependabot-quickstart-guide>, 2025, [Accessed: May 9, 2025].
- [97] "Licensing a repository - GitHub Docs," <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/licensing-a-repository>, 2025, [Accessed: May 9, 2025].
- [98] "GitLab," <https://gitlab.com/gitlab-com>, 2025, [Accessed: May 9, 2025].
- [99] "GitLab Licensing and Compatibility," <https://docs.gitlab.com/ee/development/licensing.html>, 2025, [Accessed: May 9, 2025].
- [100] "License approval policies," https://docs.gitlab.com/ee/user/compliance/license_approval_policies.html, 2025, [Accessed: May 9, 2025].
- [101] "Aqua Cloud Native Security, Container & Serverless Security," <https://www.aquasec.com/>, 2025, [Accessed: May 9, 2025].
- [102] "5 Open Source Licenses and Compliance Risks to Know About," <https://www.aquasec.com/cloud-native-academy/supply-chain-security/open-source-license/>, 2022, [Accessed: May 9, 2025].
- [103] "What Is Open Source Security?" <https://www.aquasec.com/cloud-native-academy/devsecops/open-source-security/>, 2025, [Accessed: May 9, 2025].
- [104] J. Worthington, A. DeMartine, D. Beaton, and P. Harrison, "The Forrester Wave™: Software Composition Analysis, Q2 2023," <https://www.forrester.com/report/the-forrester-wave-tm-software-composition-analysis-q2-2023/RES178483>, 2023, [Accessed: May 9, 2025].
- [105] "Github - lby2001/replication-package-of-oss-license-slr/literature-list.png," <https://github.com/LBY2001/Replication-Package-of-OSS-License-SLR/blob/master/Literature%20List.png>, 2025, [Accessed: May 9, 2025].
- [106] T. Tuunanen, J. Koskinen, and T. Kärkkäinen, "Automated software license analysis," *Automated Software Engineering*, vol. 16, pp. 455–490, 2009.
- [107] "Github - fossology/fossology/src/nomos," <https://github.com/fossology/fossology/tree/master/src/nomos>, [Accessed: May 9, 2025].
- [108] Y. Higashi, M. Ohira, Y. Kashiwa, and Y. Manabe, "Hierarchical clustering of oss license statements toward automatic generation of license rules," *Journal of information processing*, vol. 27, pp. 42–50, 2019.
- [109] Y. Higashi, M. Ohira, and Y. Manabe, "Automating license rule generation to help maintain rule-based oss license identification tools," *Journal of Information Processing*, vol. 31, pp. 2–12, 2023.
- [110] A. V. Aho and M. J. Corasick, "Efficient string matching: an aid to bibliographic search," *Commun. ACM*, vol. 18, no. 6, p. 333–340, Jun. 1975. [Online]. Available: <https://doi.org/10.1145/360825.360855>
- [111] M. Di Penta, D. M. German, and G. Antoniol, "Identifying licensing of jar archives using a code-search approach," in *2010 7th IEEE Working Conference on Mining Software Repositories*, 2010, pp. 151–160.
- [112] H. Zhang, B. Shi, and L. Zhang, "Automatic checking of license compliance," in *2010 IEEE International Conference on Software Maintenance*. IEEE, 2010, pp. 1–3.
- [113] D. Pogrebny, I. Kuznetsov, Y. Golubev, V. Tankov, and T. Bryksin, "Sorrel: an ide plugin for managing licenses and detecting license incompatibilities," in *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2021, pp. 574–578.
- [114] T. Wolter, A. Barcomb, D. Riehle, and N. Harutyunyan, "Open source license inconsistencies on github," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 5, pp. 1–23, 2023.
- [115] C. Vendome, M. Linares-Vázquez, G. Bavota, M. Di Penta, D. German, and D. Poshyvanyk, "Machine learning-based detection of open source license exceptions," in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 2017, pp. 118–129.
- [116] G. M. Kapitsaki and D. Paschalides, "Identifying terms in open source software license texts," in *2017 24th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2017, pp. 540–545.
- [117] L. Li, S. Xu, Y. Liu, Y. Gao, X. Cai, J. Wu, W. Song, and Z. Liu, "Lisum: Open source software license summarization with multi-task learning," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 787–799.
- [118] S. Xu, Y. Gao, L. Fan, L. Li, X. Cai, and Z. Liu, "LiResolver: License incompatibility resolution for open source software," in *32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 652–663.
- [119] Y. Higashi, Y. Manabe, and M. Ohira, "Clustering oss license statements toward automatic generation of license rules," in *2016 7th International Workshop on Empirical Software Engineering in Practice (IWSEEP)*, 2016, pp. 30–35.
- [120] "GitHub - fossology/fossology/wiki/Monk," <https://github.com/fossology/fossology/wiki/Monk>, [Accessed: May 9, 2025].
- [121] M. Papoutsoglou, G. M. Kapitsaki, D. German, and L. Angelis, "An analysis of open source software licensing questions in stack exchange sites," *Journal of Systems and Software*, vol. 183, p. 111113, 2022.
- [122] R. Meloca, G. Pinto, L. Baiser, M. Mattos, I. Polato, I. Wiese, and D. M. German, "Understanding the usage, impact, and adoption of non-osi approved licenses," in *2018 IEEE/ACM 15th International Conference on Mining Software Repositories*, 2018, pp. 270–280.
- [123] L. Xiao, X. Huang, B. Chen, and L. Jing, "Label-specific document representation for multi-label text classification," in *2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, 2019, pp. 466–475.

- [124] *SPDX License List*, The Linux Foundation, 2024, (Accessed on 2/08/2024). [Online]. Available: <https://spdx.org/licenses/>
- [125] P. Nordquist, A. Petersen, and A. Todorova, "License tracing in free, open, and proprietary software," *Journal of Computing Sciences in Colleges*, vol. 19, no. 2, pp. 101–112, 2003.
- [126] T. F. Gordon, "Analyzing open source license compatibility issues with carneades," in *13th International Conference on Artificial Intelligence and Law*, 2011, pp. 51–55.
- [127] N. Ilo, J. Grabner, T. Artner, M. Bernhart, and T. Grechenig, "Combining software interrelationship data across heterogeneous software repositories," in *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2015, pp. 571–575.
- [128] D. Riehle and N. Harutyunyan, "Open-source license compliance in software supply chains," in *Towards Engineering Free/Libre Open Source Software (FLOSS) Ecosystems for Impact and Sustainability: Communications of NII Shonan Meetings*. Springer, 2019, pp. 83–95.
- [129] N. Harutyunyan and D. Riehle, "Open source software governance: A case study evaluation of supply chain management best practices," 2023.
- [130] Z. Sun, Z. Quan, S. Yu, L. Zhang, and D. Mao, "A knowledge-driven framework for software supply chain security analysis," in *2024 8th International Conference on Control Engineering and Artificial Intelligence*, 2024, pp. 267–272.
- [131] D. M. German, M. Di Penta, and J. Davies, "Understanding and auditing the licensing of open source software distributions," in *2010 IEEE 18th International Conference on Program Comprehension*. IEEE, 2010, pp. 84–93.
- [132] A. Mathur, H. Choudhary, P. Vashist, W. Thies, and S. Thilagam, "An empirical study of license violations in open source projects," in *2012 35th Annual IEEE Software Engineering Workshop*. IEEE, 2012, pp. 168–176.
- [133] O. Mlouki, F. Khomh, and G. Antoniol, "On the detection of licenses violations in the android ecosystem," in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, vol. 1. IEEE, 2016, pp. 382–392.
- [134] S. Qiu, D. M. German, and K. Inoue, "Empirical study on dependency-related license violation in the javascript package ecosystem," *Journal of Information Processing*, vol. 29, pp. 296–304, 2021.
- [135] J. P. Moraes, I. Polato, I. Wiese, F. Saraiva, and G. Pinto, "From one to hundreds: multi-licensing in the javascript ecosystem," *Empirical Software Engineering*, vol. 26, pp. 1–29, 2021.
- [136] I. S. Makari, A. Zerouali, and C. De Roover, "Prevalence and evolution of license violations in npm and rubygems dependency networks," in *International Conference on Software and Software Reuse*. Springer, 2022, pp. 85–100.
- [137] A. Antelmi, M. Torquati, G. Corridori, D. Gregori, F. Polzella, G. Spinatelli, and M. Aldinucci, "Analyzing foss license usage in publicly available software at scale via the sw-h-analytics framework," *The Journal of Supercomputing*, pp. 1–35, 2024.
- [138] T. A. Alspaugh, H. U. Asuncion, and W. Scacchi, "Presenting software license conflicts through argumentation," *23rd International Conference on Software Engineering and Knowledge Engineering (SEKE 2011)*, pp. 35–40, 2011.
- [139] W. Xu, X. Wu, R. He, and M. Zhou, "Licenserec: Knowledge based open source license recommendation for oss projects," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE-Companion)*. IEEE, 2023, pp. 180–183.
- [140] O. S. A. D. L. contributors, "Osadl open source license checklists: Osadl - open source automation development lab eg," <https://www.osadl.org/OSADL-Open-Source-License-Checklists.oss-compliance-lists.0.html>, 2024, [Accessed: May 9, 2025].
- [141] "Github - osslab-pku/reclicense/backend/app/knowledgebase/compatibility_63.csv," https://github.com/osslab-pku/RecLicense/blob/master/backend/app/knowledgebase/compatibility_63.csv, [Accessed: May 9, 2025].
- [142] T. A. Alspaugh, H. U. Asuncion, and W. Scacchi, "Analyzing software licenses in open architecture software systems," in *2009 ICSE Workshop on Emerging Trends in Free/Libre/Open Source Software Research and Development*. IEEE, 2009, pp. 54–57.
- [143] T. A. Alspaugh, H. U. Asuncion, and W. Scacchi, "Intellectual property rights requirements for heterogeneously-licensed systems," in *2009 17th IEEE International Requirements Engineering Conference*. IEEE, 2009, pp. 24–33.
- [144] "Ip license agreement: Sublicensing clause," [https://uk.practicallaw.thomsonreuters.com/w-002-9143?transitionType=Default&contextData=\(sc.Default\)&firstPage=true](https://uk.practicallaw.thomsonreuters.com/w-002-9143?transitionType=Default&contextData=(sc.Default)&firstPage=true), 2025, [Accessed: May 9, 2025].
- [145] "Choose an open source license." <https://choosealicense.com/>, 2025, [Accessed: May 9, 2025].
- [146] F. Inc., "Software Licenses in Plain English," <https://tldrlegal.com/>, 2025, [Accessed: May 9, 2025].
- [147] K. Gao, Q. Ma, Z. Li, Z. Zheng, and C. Yang, "License Show Room," <https://compliance.openeuler.org/>, 2025, [Accessed: May 9, 2025].
- [148] "Copyleft - Wikipedia," <https://en.wikipedia.org/wiki/Copyleft>, 2025, [Accessed: May 9, 2025].
- [149] A. Monden, S. Okahara, Y. Manabe, and K. Matsumoto, "Guilty or not guilty: Using clone metrics to determine open source licensing violations," *IEEE software*, vol. 28, no. 2, pp. 42–47, 2010.
- [150] F. Boughanmi, "Multi-language and heterogeneously-licensed software analysis," in *2010 17th working conference on reverse engineering*. IEEE, 2010, pp. 293–296.
- [151] L. An, O. Mlouki, F. Khomh, and G. Antoniol, "Stack overflow: A code laundering platform?" in *2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2017, pp. 283–293.
- [152] A. Lotter, S. A. Licorish, B. T. R. Savarimuthu, and S. Meldrum, "Code reuse in stack overflow and popular open source java projects," in *2018 25th Australasian Software Engineering Conference (ASWEC)*. IEEE, 2018, pp. 141–150.
- [153] S. Baltes and S. Diehl, "Usage and attribution of stack overflow code snippets in github projects," *Empirical Software Engineering*, vol. 24, no. 3, pp. 1259–1295, 2019.
- [154] C. Ragkhitwetsagul, J. Krinke, M. Paixao, G. Bianco, and R. Oliveto, "Toxic code snippets on stack overflow," *IEEE Transactions on Software Engineering*, vol. 47, no. 3, pp. 560–581, 2019.
- [155] Y. Golubev, M. Eliseeva, N. Povarov, and T. Bryksin, "A study of potential code borrowing and license violations in java projects on github," in *17th International Conference on Mining Software Repositories*, 2020, pp. 54–64.
- [156] D. Serafini and S. Zacchiroli, "Efficient prior publication identification for open source code," in *18th International Symposium on Open Collaboration*, 2022, pp. 1–8.
- [157] M. Ciniselli, L. Pascarella, and G. Bavota, "To what extent do deep learning-based code recommenders generate predictions by cloning code from the training set?" in *19th International Conference on Mining Software Repositories*, 2022, pp. 167–178.
- [158] Z. Yu, Y. Wu, N. Zhang, C. Wang, Y. Vorobeychik, and C. Xiao, "Codeiprompt: intellectual property infringement assessment of code language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 40373–40389.
- [159] G. M. Kapitsaki, "Generative ai for code generation: Software reuse implications," in *International Conference on Software and Software Reuse*. Springer, 2024, pp. 37–47.
- [160] M. Duan, Q. Li, and B. He, "Modelgo: A practical tool for machine learning license analysis," in *ACM on Web Conference 2024*, 2024, pp. 1158–1169.
- [161] A. J. M. Molina and T. Shinohara, "Fast approximate matching of programs for protecting libre/open source software by using spatial indexes," in *Seventh IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2007, pp. 111–122.
- [162] A. Hemel, K. T. Kalleberg, R. Vermaas, and E. Dolstra, "Finding software license violations through binary code clone detection," in *8th Working Conference on Mining Software Repositories*, 2011, pp. 63–72.
- [163] M. Feng, W. Mao, Z. Yuan, Y. Xiao, G. Ban, W. Wang, S. Wang, Q. Tang, J. Xu, H. Su *et al.*, "Open-source license violations of binary software at large scale," in *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2019, pp. 564–568.
- [164] D. Zhang, P. Luo, W. Tang, and M. Zhou, "Oslsdetector: Identifying open-source libraries through binary analysis," in *35th IEEE/ACM International Conference on Automated Software Engineering*, 2020, pp. 1312–1315.
- [165] M. Di Penta and D. M. German, "Who are source code contributors and how do they change?" in *2009 16th Working Conference on Reverse Engineering*. IEEE, 2009, pp. 11–20.
- [166] M. Di Penta, D. M. German, Y.-G. Guéhéneuc, and G. Antoniol, "An exploratory study of the evolution of software licensing," in *32nd ACM/IEEE International Conference on Software Engineering-Volume 1*, 2010, pp. 145–154.
- [167] C. Vendome, M. Linares-Vásquez, G. Bavota, M. Di Penta, D. German, and D. Poshyvanyk, "License usage and changes: a large-scale study of java projects on github," in *2015 IEEE 23rd International Conference on Program Comprehension*. IEEE, 2015, pp. 218–228.

- [168] C. Vendome, G. Bavota, M. D. Penta, M. Linares-Vásquez, D. German, and D. Poshyvanyk, "License usage and changes: a large-scale study on github," *Empirical Software Engineering*, vol. 22, pp. 1537–1577, 2017.
- [169] D. Reid and A. Mockus, "Applying the universal version history concept to help de-risk copy-based code reuse," in *2023 IEEE 23rd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2023, pp. 1–12.
- [170] "Stack Exchange and Stack Overflow have moved to CC BY-SA 4.0," <https://meta.stackexchange.com/questions/333089/stack-exchange-and-stack-overflow-have-moved-to-cc-by-sa-4-0>, Sep. 2019, [Accessed: May 9, 2025].
- [171] I. Hammouda, T. Mikkonen, V. Oksanen, and A. Jaaksi, "Open source legality patterns: architectural design decisions motivated by legal concerns," in *14th International Academic MindTrek Conference: Envisioning Future Media Environments*, 2010, pp. 207–214.
- [172] W. Scacchi and T. A. Alspaugh, "Understanding the role of licenses and evolution in open architecture software ecosystems," *Journal of Systems and Software*, vol. 85, no. 7, pp. 1479–1494, 2012.
- [173] R. Visur, "A floss license-selection methodology for cloud computing projects," in *International Conference on Cloud Computing and Services Science*, vol. 2. SCITEPRESS, 2016, pp. 129–136.
- [174] C. Vendome and D. Poshyvanyk, "Assisting developers with license compliance," in *38th International Conference on Software Engineering Companion*, 2016, pp. 811–814.
- [175] G. M. Kapitsaki and G. Charalambous, "Modeling and recommending open source licenses with findosslicense," *IEEE Transactions on Software Engineering*, vol. 47, no. 5, pp. 919–935, 2019.
- [176] K. Huang, Y. Xia, B. Chen, S. He, H. Zeng, Z. Zhou, J. Guo, and X. Peng, "Your 'notice' is missing: Detecting and fixing violations of modification terms in open source licenses during forking," in *33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1022–1034.
- [177] J. A. Colazo, Y. Fang, and D. Neufeld, "Development success in open source software projects: Exploring the impact of copylefted licenses," in *Americas Conference on Information Systems (AMCIS)*, 2005, p. 432.
- [178] K. J. Stewart, A. P. Ammeter, and L. M. Maruping, "A preliminary analysis of the influences of licensing and organizational sponsorship on success in open source projects," in *38th Annual Hawaii International Conference on System Sciences*. IEEE, 2005, pp. 197c–197c.
- [179] J. Colazo and Y. Fang, "Impact of license choice on open source software development activity," *Journal of the American Society for Information Science and Technology*, vol. 60, no. 5, pp. 997–1011, 2009.
- [180] J. Lindman, M. Rossi, and A. Paajanen, "Matching open source software licenses with corresponding business models," *IEEE software*, vol. 28, no. 4, pp. 31–35, 2011.
- [181] Y. Kashima, Y. Hayase, N. Yoshida, Y. Manabe, and K. Inoue, "An investigation into the impact of software licenses on copy-and-paste reuse among oss projects," in *2011 18th Working Conference on Reverse Engineering*. IEEE, 2011, pp. 28–32.
- [182] C. Vendome, M. Linares-Vásquez, G. Bavota, M. Di Penta, D. M. German, and D. Poshyvanyk, "When and why developers adopt and change software licenses," in *2015 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 2015, pp. 31–40.
- [183] P. K. Medappa and S. C. Srivastava, "License choice and the changing structures of work in organization owned open source projects," in *2017 ACM SIGMIS Conference on Computers and People Research*, 2017, pp. 117–123.
- [184] J. Gamalielsson and B. Lundell, "On licensing and other conditions for contributing to widely used open source projects: an exploratory analysis," in *13th International Symposium on Open Collaboration*, 2017, pp. 1–14.
- [185] X. Zhang, H. Xu, Q. Yu, S. Zeng, S. Dai, H. Yang, and S. Wu, "License recommendation for open source projects in the power industry," *Information and Software Technology*, vol. 167, p. 107391, 2024.
- [186] H. Gu, H. He, and M. Zhou, "Self-admitted library migrations in java, javascript, and python packaging ecosystems: A comparative study," in *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2023, pp. 627–638.
- [187] T. Laverigne, O. Cappé, and F. Yvon, "Practical very large scale crfs," in *48th Annual Meeting of the Association for Computational Linguistics*, 2010, pp. 504–513.
- [188] C.-C. Chang and C.-J. Lin, "Libsvm: a library for support vector machines," *ACM transactions on intelligent systems and technology (TIST)*, vol. 2, no. 3, pp. 1–27, 2011.
- [189] "Report of license proliferation committee and draft faq," <https://open-source.org/proliferation-report>, 2025, [Accessed: May 9, 2025].
- [190] J. M. Gonzalez-Barahona, S. Montes-Leon, G. Robles, and S. Zacchiroli, "The software heritage license dataset (2022 edition)," *Empirical Software Engineering*, vol. 28, no. 6, p. 147, 2023.
- [191] OSADL, "OSADL Open Source License Checklists," <https://www.osadl.org/Access-to-raw-data.oss-compliance-raw-data-access.0.html>, [Accessed: May 9, 2025].
- [192] Z. Liu, X. Liu, Y. Zhang, Z. Zhang, S. Li, W. Niu, Q. Zhou, R. Zhou, and X. Zhou, "Liscopelens: An open-source license incompatibility analysis tool based on scope representation of license terms," in *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2024, pp. 13–24.
- [193] Q. Ke, X. Hou, Y. Zhao, and H. Wang, "Clausebench: Enhancing software license analysis with clause-level benchmarking," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025.
- [194] T. Kamiya, S. Kusumoto, and K. Inoue, "Ccfinder: A multilingual token-based code clone detection system for large scale source code," *IEEE transactions on software engineering*, vol. 28, no. 7, pp. 654–670, 2002.
- [195] H. Sajjani, V. Saini, J. Svajlenko, C. K. Roy, and C. V. Lopes, "Sourcecerc: Scaling code clone detection to big-code," in *38th international conference on software engineering*, 2016, pp. 1157–1168.
- [196] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A novel neural source code representation based on abstract syntax tree," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 783–794.
- [197] X. Zhan, L. Fan, S. Chen, F. We, T. Liu, X. Luo, and Y. Liu, "Atvhunter: Reliable version detection of third-party libraries for vulnerability identification in android applications," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1695–1707.
- [198] "GNU Lesser General Public License version 3," <https://opensource.org/licenses/lgpl-3-0>, 2007, [Accessed: May 9, 2025].
- [199] "Google Maps Platform Terms of Service," <https://cloud.google.com/maps-platform/terms>, May 2020, [Accessed: May 9, 2025].
- [200] "Terms and conditions: This is the Android Software Development Kit License Agreement," <https://developer.android.com/studio/terms>, Jul. 2021, [Accessed: May 9, 2025].
- [201] "BSD 3-Clause No Nuclear License," <https://spdx.org/licenses/BSD-3-Clause-No-Nuclear-License.html>, [Accessed: May 9, 2025].
- [202] "The JSON License," <https://www.json.org/license.html>, [Accessed: May 9, 2025].
- [203] "Relicensing - LLVM Foundation," <https://llvm.org/docs/DeveloperPolicy.html#relicensing>, [Accessed: May 9, 2025].
- [204] "LLAMA 3.2 COMMUNITY LICENSE AGREEMENT," https://www.llama.com/llama3_2/license/, 2024, [Accessed: May 9, 2025].
- [205] "GitHub - deepseek-ai/DeepSeek-Coder/LICENCE-MODEL," <https://github.com/deepseek-ai/deepseek-coder/blob/main/LICENSE-MODEL>, 2023, [Accessed: May 9, 2025].
- [206] A. Coopersmith, "The difference between Xorg and XFree86," <https://blogs.oracle.com/solaris/post/the-difference-between-xorg-and-xfree86>, 2004, [Accessed: May 9, 2025].
- [207] "Acquisition of Sun Microsystems by Oracle Corporation - Wikipedia," https://en.wikipedia.org/wiki/Acquisition_of_Sun_Microsystems_by_Oracle_Corporation#OpenOffice_resignations_and_forks, [Accessed: May 9, 2025].
- [208] "Contributor License Agreement - Wikipedia," https://en.wikipedia.org/wiki/Contributor_License_Agreement, [Accessed: May 9, 2025].
- [209] "Developer Certificate of Origin - Wikipedia," https://en.wikipedia.org/wiki/Developer_Certificate_of_Origin, [Accessed: May 9, 2025].
- [210] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," *ICLR*, 2023.
- [211] "Github copilot · your ai pair programmer," <https://github.com/features/copilot>, 2025, [Accessed: May 9, 2025].
- [212] "Chatgpt," <https://chatgpt.com/>, 2025, [Accessed: May 9, 2025].
- [213] GitHub, "GitHub Trust Center," <https://resources.github.com/copilot-trust-center>, 2025, [Accessed: May 9, 2025].
- [214] W. Xu, K. Gao, H. He, and M. Zhou, "Licoeval: Evaluating llms on license compliance in code generation," *arXiv preprint arXiv:2408.02487*, 2024.
- [215] "Participate in Our Community - OpenChain," <https://openchainproject.org/participate>, 2025, [Accessed: May 9, 2025].